



INVESTIGANDO PROBLEMAS DE ESCALABILIDADE DE E/S NO ALGORITMO SDDP EM AMBIENTES HPC

Marcelo Barros da Cruz

Dissertação de Mestrado apresentada ao Programa de Pós-graduação em Engenharia de Sistemas e Computação, COPPE, da Universidade Federal do Rio de Janeiro, como parte dos requisitos necessários à obtenção do título de Mestre em Engenharia de Sistemas e Computação.

Orientador: Nome do Orientador Sobrenome

Rio de Janeiro
Abril de 2026

INVESTIGANDO PROBLEMAS DE ESCALABILIDADE DE E/S NO
ALGORITMO SDDP EM AMBIENTES HPC

Marcelo Barros da Cruz

DISSERTAÇÃO SUBMETIDA AO CORPO DOCENTE DO INSTITUTO
ALBERTO LUIZ COIMBRA DE PÓS-GRADUAÇÃO E PESQUISA DE
ENGENHARIA DA UNIVERSIDADE FEDERAL DO RIO DE JANEIRO
COMO PARTE DOS REQUISITOS NECESSÁRIOS PARA A OBTENÇÃO DO
GRAU DE MESTRE EM CIÊNCIAS EM ENGENHARIA DE SISTEMAS E
COMPUTAÇÃO.

Orientador: Nome do Orientador Sobrenome

Aprovada por: Prof. Nome do Primeiro Examinador Sobrenome
Prof. Nome do Segundo Examinador Sobrenome
Prof. Nome do Terceiro Examinador Sobrenome

RIO DE JANEIRO, RJ – BRASIL
ABRIL DE 2026

Barros da Cruz, Marcelo

Investigando Problemas de Escalabilidade de E/S no Algoritmo SDDP em Ambientes HPC/Marcelo Barros da Cruz. – Rio de Janeiro: UFRJ/COPPE, 2026.

XIII, 40 p.: il.; 29, 7cm.

Orientador: Nome do Orientador Sobrenome

Dissertação (mestrado) – UFRJ/COPPE/Programa de Engenharia de Sistemas e Computação, 2026.

Referências Bibliográficas: p. 38 – 40.

1. MPI-IO. 2. Lustre. 3. SDDP. 4. Computação de Alto Desempenho. 5. Escalabilidade de E/S. I. Sobrenome, Nome do Orientador. II. Universidade Federal do Rio de Janeiro, COPPE, Programa de Engenharia de Sistemas e Computação. III. Título.

*Dedico esta dissertação a [a
definir].*

Agradecimentos

ToDp: redigir agradecimentos finais. Coloque aqui seus agradecimentos.

Resumo da Dissertação apresentada à COPPE/UFRJ como parte dos requisitos necessários para a obtenção do grau de Mestre em Ciências (M.Sc.)

INVESTIGANDO PROBLEMAS DE ESCALABILIDADE DE E/S NO ALGORITMO SDDP EM AMBIENTES HPC

Marcelo Barros da Cruz

Abril/2026

Orientador: Nome do Orientador Sobrenome

Programa: Engenharia de Sistemas e Computação

A escalabilidade eficiente de E/S continua sendo um desafio crítico em aplicações paralelas de dados massivos, especialmente em sistemas de alto desempenho que lidam com grandes volumes de dados. Embora avanços como o uso de MPI-IO e sistemas de arquivos paralelos como o Lustre tenham contribuído significativamente, muitas aplicações ainda enfrentam gargalos que comprometem a performance. Esse problema é particularmente evidente em modelos de otimização utilizados no setor energético, como o SDDP, que demandam acesso frequente e distribuído a grandes conjuntos de dados. A eficiência da E/S torna-se, assim, um fator decisivo para a viabilidade e o desempenho de simulações complexas em ambientes paralelos. Esta dissertação investiga os gargalos de E/S do algoritmo SDDP e propõe uma arquitetura MPI-IO baseada em MPI-IO sobre Lustre, avaliada empiricamente nos ambientes AWS (FSx for Lustre) e Santos Dumont (LNCC).

Abstract of Dissertation presented to COPPE/UFRJ as a partial fulfillment of the requirements for the degree of Master of Science (M.Sc.)

INVESTIGATING I/O SCALABILITY ISSUES IN THE SDDP ALGORITHM ON HPC ENVIRONMENTS

Marcelo Barros da Cruz

April/2026

Advisor: Nome do Orientador Sobrenome

Department: Systems Engineering and Computer Science

Efficient I/O scalability remains a major challenge for parallel applications handling massive data volumes, particularly in high-performance computing environments. Although advances such as MPI-IO and parallel file systems like Lustre have significantly improved performance, many applications still face I/O bottlenecks that hinder scalability. This issue is particularly critical in energy sector optimization models, such as Stochastic Dual Dynamic Programming (SDDP), which require frequent and distributed access to large datasets. Therefore, optimizing I/O operations becomes essential to ensure the feasibility and efficiency of complex simulations in parallel computing systems. This dissertation investigates the I/O bottlenecks of the SDDP algorithm and proposes an MPI-IO-based architecture over Lustre, empirically evaluated on AWS (FSx for Lustre) and on the Santos Dumont supercomputer (LNCC).

Sumário

Lista de Figuras	x
Lista de Tabelas	xii
Lista de Abreviaturas	xiii
1 Introdução	1
2 Fundamentação Teórica	3
2.1 MPI, protocolos de comunicação e operações paralelas de arquivos . .	3
2.2 Sistema de arquivos paralelo Lustre	7
2.3 Adaptadores de rede em HPC: Ethernet e InfiniBand	7
2.4 Computação em nuvem para HPC: AWS	8
2.5 O Programa Mensal da Operação (PMO)	8
2.6 Algoritmo SDDP	9
2.6.1 Padrão dos dados de saída	10
2.6.2 Arquitetura atual de E/S do SDDP	12
3 Paralelização dos Mecanismos de E/S do SDDP	14
3.1 Uso de MPI-IO com Lustre para escalabilidade de E/S	14
3.2 Organização dos arquivos compartilhados de resultados	16
4 Avaliações	18
4.1 Experimento SDumont	19
4.1.1 Avaliação da implementação atual	20
4.1.2 Avaliação da implementação MPI-IO com escritas independentes	21
4.1.3 Análise comparativa	21
4.2 Experimento AWS	27
4.2.1 Avaliação da implementação atual	27
4.2.2 Avaliação da implementação MPI-IO com escritas independentes	28
4.2.3 Análise comparativa	29
4.2.4 Avaliação do impacto do número de OSTs	33

5	Trabalhos relacionados	36
6	Conclusões e trabalhos futuros	37
	Referências Bibliográficas	38

Lista de Figuras

2.1	Relação entre comunicação ponto a ponto e E/S paralela. No modelo centralizado, os resultados passam por um processo escritor; no modelo MPI-IO, os próprios <i>ranks</i> escrevem diretamente no arquivo.	4
2.2	Diferença entre escritas independentes e coletivas em MPI-IO. Escritas independentes preservam autonomia dos <i>ranks</i> ; escritas coletivas permitem agregação e reordenação das requisições pela biblioteca MPI-IO.	6
2.3	Estrutura de cenários e estágios do SDDP. Cada célula corresponde a um subproblema (estágio, cenário). Os cenários são mutuamente independentes e podem ser distribuídos entre <i>ranks</i> MPI, caracterizando uma aplicação <i>bag-of-tasks</i>	10
2.4	Exemplo de arquivo de saída por patamar de carga do SDDP com cinco patamares por estágio e cenário.	11
2.5	Exemplo de arquivo de saída horário do SDDP com 720 horas por estágio e colunas de valores para agentes $A_1, \dots, A_N$	12
2.6	Algoritmo SDDP com processo escritor.	12
3.1	Escrita independente dos processos SDDP em um arquivo compartilhado e distribuição dos pedaços em OSTs do Lustre.	17
3.2	Proposta com MPI-IO e Lustre.	17
4.1	Distribuição agregada da quantidade de blocos de dados por faixa de tamanho no experimento, em escala logarítmica.	19
4.2	Banda agregada média das implementações atual e MPI-IO no Experimento SDumont.	23
4.3	Decomposição do tempo médio por processo das implementações atual e MPI-IO no Experimento SDumont.	24
4.4	Tempo médio de bloqueio das operações de envio por tamanho de bloco de dados nas implementações atual e MPI-IO no Experimento SDumont (escala logarítmica).	26
4.5	Banda agregada média no Experimento AWS.	30

4.6	Tempo de execução no Experimento AWS.	32
4.7	Tempo médio de bloqueio das operações de envio em função do tamanho dos blocos de dados no Experimento AWS (escala logarítmica).	33
4.8	Tempo médio de envio por classe de tamanho das mensagens para configurações MPI-IO com 4 e 10 OSTs.	34
4.9	Banda agregada média para configurações MPI-IO com 4 e 10 OSTs em 16 e 32 nós.	35

Lista de Tabelas

2.1	Critérios práticos para escolha do modo de escrita em MPI-IO.	7
4.1	Desempenho do Experimento SDumont com a implementação atual. O tempo total corresponde à soma do tempo médio de simulação e comunicação; o tempo de I/O é uma parcela interna da simulação e não é somado novamente. A eficiência é calculada em relação à base de 2 nós.	20
4.2	Desempenho do Experimento SDumont com a implementação MPI- IO. O tempo total corresponde à soma do tempo médio de simulação e comunicação; o tempo de I/O é uma parcela interna da simulação e não é somado novamente. A eficiência é calculada em relação à base de 2 nós.	21
4.3	Resumo comparativo entre a implementação atual e MPI-IO no Ex- perimento SDumont. A melhoria no total e a melhoria de I/O são calculadas em relação à implementação atual; valores negativos indi- cam piora.	22
4.4	Desempenho do Experimento AWS com a implementação atual, in- cluindo tempos de I/O, comunicação, tempo total, <i>speedup</i> , eficiência e percentual de I/O. O total corresponde ao tempo de simulação mais comunicação, e a eficiência é calculada em relação à base de 2 nós. . .	28
4.5	Desempenho do Experimento AWS com a implementação MPI-IO, incluindo tempos de I/O, comunicação, tempo total, <i>speedup</i> , eficiên- cia e percentual de I/O. O total corresponde ao tempo de simulação mais comunicação, e a eficiência é calculada em relação à base de 2 nós.	28
4.6	Resumo comparativo entre a implementação atual e MPI-IO no Expe- rimento AWS. A melhoria no total e a melhoria de I/O são calculadas em relação à implementação atual; valores negativos indicam piora. .	29

Lista de Abreviaturas

AWS	Amazon Web Services	3
E/S	Entrada/Saída	1
EFA	Elastic Fabric Adapter	3
FSx	Amazon FSx for Lustre	3
HPC	High-Performance Computing	1
LNCC	Laboratório Nacional de Computação Científica	3
MDS	Metadata Server	3
MDT	Metadata Target	3
MPI	Message Passing Interface	1
MPI-IO	MPI Input/Output	3
ONS	Operador Nacional do Sistema Elétrico	3
OSS	Object Storage Server	3
OST	Object Storage Target	3
PMO	Programa Mensal da Operação	3
RDMA	Remote Direct Memory Access	3
SDDP	Stochastic Dual Dynamic Programming	1
SIN	Sistema Interligado Nacional	3
SINAPAD	Sistema Nacional de Processamento de Alto Desempenho	3

Capítulo 1

Introdução

O aumento da complexidade e do volume de dados em aplicações científicas e de engenharia tem tornado a computação paralela uma ferramenta indispensável para atender às demandas de desempenho KANG *et al.* (2020). Entretanto, operações de entrada e saída (E/S) continuam sendo um dos principais gargalos em ambientes de alto desempenho (HPC), particularmente em aplicações que processam dados massivos LUU *et al.* (2015); XIE *et al.* (2020). Nesse contexto, o uso de bibliotecas como o MPI-IO, combinadas com sistemas de arquivos paralelos como o Lustre, tornou-se uma abordagem fundamental para lidar com a escalabilidade e a eficiência no acesso aos dados MESSAGE PASSING INTERFACE FORUM (2025); OPENSFS AND EOFS (2025).

Paralelamente, a crescente inserção de fontes renováveis na matriz energética, como solar e eólica, tem introduzido novas dificuldades na formulação e resolução de problemas de otimização associados à operação dos sistemas elétricos. A variabilidade e incerteza dessas fontes requerem modelos computacionais mais complexos e dependentes de grandes volumes de dados, cuja execução eficiente depende tanto do paralelismo computacional quanto de soluções eficazes de E/S CONEJO *et al.* (2010); ZAKARIA *et al.* (2020).

ToDp: precisamos expandir esse parágrafo, explicar o que implica essa mudança de paradigma na simulação tanto para o uso de recursos computacionais como para o uso dos resultados pelos usuários do SDDP.

Assim como outras aplicações baseadas em tarefas independentes, o modelo SDDP (*Stochastic Dual Dynamic Programming*) é amplamente utilizado para planejamento e despacho hidrotérmico em diversos países. À medida que os modelos se tornam mais detalhados, em resolução horária, e a quantidade de cenários aumenta, a execução do SDDP passa a demandar não apenas alto poder computacional, mas também operações intensivas de E/S PEREIRA e PINTO (1991).

Nesse contexto, a análise do desempenho do MPI-IO sobre sistemas de arquivos paralelos como o Lustre, aplicada a esse tipo de aplicação, representa uma contribu-

ição significativa para o desenvolvimento de soluções mais robustas e escaláveis em ambientes HPC.

ToDp: Apresentar organização da dissertação ao final da introdução (capítulos 2–5).

Capítulo 2

Fundamentação Teórica

2.1 MPI, protocolos de comunicação e operações paralelas de arquivos

O *Message Passing Interface* (MPI) é o padrão dominante para programação paralela baseada em processos que se comunicam por troca de mensagens. Em aplicações científicas executadas em *clusters* e supercomputadores, o MPI fornece tanto primitivas de comunicação ponto a ponto, como `MPI_Send` e `MPI_Recv`, quanto operações coletivas, como reduções, barreiras e difusões. A partir do MPI-2, o padrão também passou a incluir a especificação MPI-IO, que estende o mesmo modelo de coordenação entre processos para operações paralelas de entrada e saída em arquivos MESSAGE PASSING INTERFACE FORUM (2023, 1997).

Essa integração é importante porque comunicação e E/S não são problemas independentes em aplicações HPC. Uma arquitetura que concentra resultados em um único processo escritor transforma a E/S em comunicação ponto a ponto: os processos produtores enviam seus dados para um *rank* central, e esse *rank* serializa a escrita no sistema de arquivos. Por outro lado, uma arquitetura baseada em MPI-IO permite que os próprios processos produtores escrevam seus resultados, deslocando o gargalo do processo escritor para o sistema de arquivos paralelo. A escolha entre essas estratégias depende do tamanho das mensagens, do padrão de acesso aos dados, da regularidade dos deslocamentos no arquivo e da capacidade do sistema de armazenamento de atender requisições concorrentes.

No contexto da comunicação ponto a ponto, o comportamento de uma chamada de envio não é determinado apenas pela semântica da função usada pela aplicação, mas também pelo protocolo escolhido internamente pela biblioteca. Dois protocolos são particularmente relevantes: *eager* e *rendezvous*. No protocolo *eager*, a implementação envia pequenos blocos de dados de forma imediata, normalmente copiando-os para *buffers* internos até que o receptor execute a chamada correspondente. Esse

caminho favorece baixa latência para mensagens pequenas, pois evita uma etapa prévia de negociação, mas consome memória proporcional ao número de mensagens em trânsito.

No protocolo *rendezvous*, usado tipicamente para blocos maiores, o emissor primeiro negocia com o receptor antes da transferência efetiva dos dados. Esse *handshake* aumenta a latência inicial, porém reduz a necessidade de *buffers* intermediários e torna o consumo de memória mais previsível em cargas intensivas GROPP *et al.* (1999); THAKUR *et al.* (2005). Assim, mensagens pequenas e frequentes tendem a se beneficiar do *eager*, enquanto mensagens grandes favorecem o *rendezvous*, especialmente quando a aplicação opera próxima aos limites de memória ou de rede.

A Figura 2.1 resume essa relação entre comunicação ponto a ponto e E/S paralela. O fluxo centralizado usa a rede para mover os resultados até um escritor único; o fluxo MPI-IO remove essa etapa intermediária e permite que os processos escrevam diretamente em regiões distintas do arquivo.

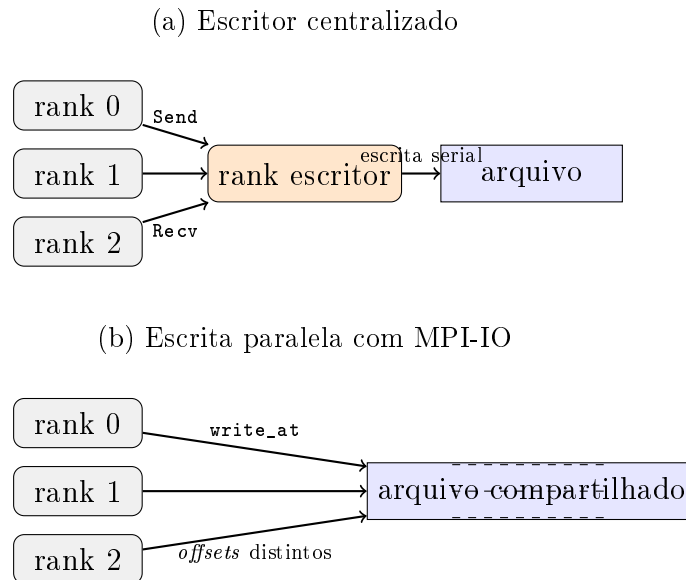


Figura 2.1: Relação entre comunicação ponto a ponto e E/S paralela. No modelo centralizado, os resultados passam por um processo escritor; no modelo MPI-IO, os próprios *ranks* escrevem diretamente no arquivo.

Para operações paralelas de arquivos, o MPI-IO define uma API padronizada para que múltiplos processos acessem arquivos compartilhados de forma coordenada. As operações podem ser classificadas, de maneira prática, em três grupos: escritas independentes, escritas coletivas e escritas não bloqueantes ou assíncronas. Cada grupo tem custos e benefícios distintos, e a escolha correta depende do padrão de dados produzido pela aplicação.

Do ponto de vista do arquivo, dois padrões de acesso são particularmente relevantes: acesso contíguo e acesso com *strides*. No acesso contíguo, cada processo

lê ou escreve uma região contínua do arquivo, como um bloco de resultados armazenado sequencialmente. Esse padrão é favorável ao sistema de arquivos paralelo, pois tende a gerar requisições maiores, alinhadas e com menor sobrecarga de metadados. No acesso com *strides*, por outro lado, os dados de um processo aparecem em regiões separadas por intervalos regulares, como ocorre em matrizes distribuídas por linhas, colunas ou blocos intercalados. Embora esse padrão represente de forma natural muitas estruturas científicas, ele pode produzir várias requisições pequenas e espaçadas quando tratado de maneira ingênua. O MPI-IO permite descrever esses acessos não contíguos por meio de tipos derivados e *file views*, possibilitando que a biblioteca reorganize a E/S por técnicas como *data sieving* e *two-phase I/O* MESSAGE PASSING INTERFACE FORUM (2023); ROSS *et al.* (2010); THAKUR *et al.* (1999a).

Nas escritas independentes, cada processo executa sua própria chamada de E/S sem exigir que os demais processos participem da mesma operação. Um exemplo típico é `MPI_File_write_at`, no qual o processo informa o *offset* do arquivo em que seus dados devem ser gravados. Esse padrão é adequado quando cada *rank* produz blocos naturalmente independentes, quando os tamanhos variam entre processos ou quando a aplicação não possui pontos convenientes de sincronização global. Também é uma escolha simples e robusta para aplicações do tipo *bag-of-tasks* CIRNE *et al.* (2003), em que tarefas independentes terminam em momentos diferentes e podem persistir seus resultados sem aguardar as demais.

O custo das escritas independentes é que o sistema de arquivos pode receber muitas requisições pequenas, desalinhadas ou concorrentes. Em sistemas paralelos como o Lustre, isso pode funcionar bem quando os blocos são suficientemente grandes e os *offsets* estão bem distribuídos entre *stripes* e OSTs. Entretanto, quando muitos processos escrevem pequenas regiões dispersas, a sobrecarga de metadados e a contenção nos servidores de armazenamento podem limitar a escalabilidade.

Nas escritas coletivas, todos os processos de um comunicador participam da mesma operação, como em `MPI_File_write_all`. Nesse caso, a biblioteca MPI-IO tem uma visão global dos acessos e pode reorganizar a E/S por meio de técnicas como *two-phase I/O*, agregação de requisições e *data sieving*. Em vez de cada processo enviar pequenas escritas diretamente ao sistema de arquivos, alguns processos agregadores podem reunir dados de vários *ranks* e emitir operações maiores, mais contíguas e melhor alinhadas ao particionamento do arquivo.

Esse padrão é recomendado quando a aplicação possui fases bem definidas de produção de dados, quando todos ou quase todos os processos escrevem em uma mesma etapa e quando os acessos formam uma estrutura regular no arquivo, como matrizes distribuídas, campos de simulação em grades, *checkpoints* globais e saídas por passo de tempo. A escrita coletiva tende a reduzir o número de chamadas ao

sistema de arquivos e pode aumentar a banda efetiva, mas introduz sincronização entre processos. Se alguns *ranks* produzem pouco dado, atrasam a chegada à chamada coletiva ou têm padrões de acesso muito irregulares, a operação coletiva pode aumentar o tempo de espera e prejudicar a sobreposição com a computação.

O MPI-IO também oferece variantes não bloqueantes, como `MPI_File_istore_at` e `MPI_File_istore_all`. Nessas operações, a chamada inicia a E/S e retorna antes de sua conclusão, permitindo que a aplicação execute computação ou comunicação enquanto a escrita progride. A conclusão é verificada posteriormente por chamadas como `MPI_Wait` ou `MPI_Test`. Esse modelo é útil quando há trabalho computacional independente logo após a geração dos dados, pois permite sobrepor parte do custo de E/S com computação. Contudo, operações assíncronas não eliminam o custo da escrita: elas apenas mudam o ponto em que a aplicação espera por sua conclusão. Além disso, a aplicação precisa preservar os *buffers* até o término da operação e controlar o número de escritas pendentes para evitar pressão excessiva de memória ou filas de E/S.

A Figura 2.2 contrasta os dois padrões principais de escrita. Na escrita independente, cada *rank* emite sua própria operação diretamente para o arquivo. Na escrita coletiva, a biblioteca MPI-IO coordena os processos e pode transformar várias requisições pequenas em operações agregadas.

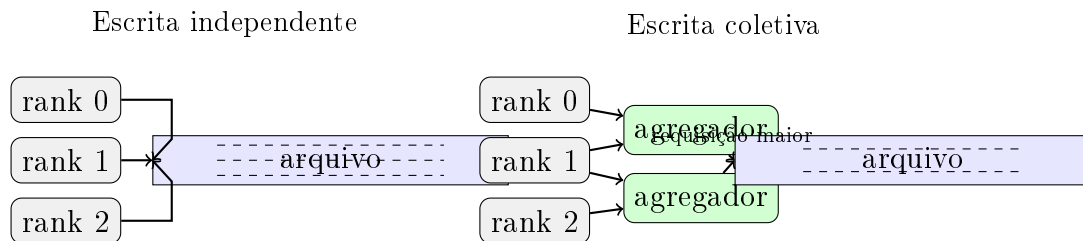


Figura 2.2: Diferença entre escritas independentes e coletivas em MPI-IO. Escritas independentes preservam autonomia dos *ranks*; escritas coletivas permitem agregação e reordenação das requisições pela biblioteca MPI-IO.

Tabela 2.1: Critérios práticos para escolha do modo de escrita em MPI-IO.

Modo de escrita	Padrão de aplicação mais adequado
Independente	Tarefas independentes, tamanhos de dados variáveis, término assíncrono dos <i>ranks</i> , <i>offsets</i> conhecidos, blocos contíguos por processo e baixa necessidade de sincronização global.
Coletiva	Fases globais de saída, dados regulares ou quase regulares, muitos acessos pequenos ou com <i>strides</i> que podem ser agregados, <i>checkpoints</i> e matrizes ou campos distribuídos.
Assíncrona independente	Aplicações com trabalho local após a geração dos dados e escritas por <i>rank</i> que podem progredir em segundo plano sem exigir participação imediata dos demais processos.
Assíncrona coletiva	Aplicações com fases coletivas bem definidas e computação útil após o início da E/S, desde que os <i>buffers</i> possam permanecer válidos até a conclusão da operação.

2.2 Sistema de arquivos paralelo Lustre

Projetado especificamente para cargas massivas de HPC, ele distribui os dados em *Object Storage Targets* (OSTs) e os metadados em *Metadata Servers* (MDS), permitindo acesso paralelo e balanceado a múltiplos clientes. Além disso, o Lustre utiliza a técnica de *striping*, na qual um arquivo é dividido em blocos (*chunks* ou *stripes*) distribuídos entre diferentes OSTs. O tamanho do *stripe* e o número de OSTs utilizados podem ser configurados de acordo com o padrão de acesso da aplicação, influenciando diretamente no *throughput* e na escalabilidade: *stripes* maiores tendem a favorecer grandes operações sequenciais, enquanto *stripes* menores podem beneficiar acessos concorrentes e distribuídos BRAAM (2002). Graças a essa flexibilidade e alto desempenho, o Lustre tornou-se amplamente adotado em supercomputadores listados no TOP500, constituindo um elemento-chave para aplicações científicas e industriais que exigem desempenho extremo em E/S.

2.3 Adaptadores de rede em HPC: Ethernet e InfiniBand

Os adaptadores de rede são componentes críticos para o desempenho em sistemas de computação de alto desempenho, sendo a Ethernet e o InfiniBand as tecnologias mais utilizadas. A Ethernet, tradicionalmente projetada para redes locais corpora-

tivas, evoluiu para padrões de alta velocidade, como 40/100/400 Gbps, oferecendo elevada taxa de transferência (*bandwidth*), embora apresente latências relativamente maiores, frequentemente na ordem de dezenas de microssegundos. Já o InfiniBand foi desenvolvido especificamente para ambientes de HPC, combinando altas larguras de banda — que podem ultrapassar 400 Gbps em implementações modernas — com latências extremamente baixas, tipicamente abaixo de 2 μ s, graças a recursos como comunicação RDMA (*Remote Direct Memory Access*). Essa diferença torna o InfiniBand preferido em supercomputadores e *clusters* científicos, onde a baixa latência na troca de dados entre processos é tão relevante quanto a banda disponível, enquanto a Ethernet, pelo menor custo e ampla compatibilidade, mantém espaço em sistemas híbridos e *datacenters* PETRINI *et al.* (2003); GRAHAM *et al.* (2005).

2.4 Computação em nuvem para HPC: AWS

A Amazon Web Services (AWS) tem se consolidado como uma das principais plataformas de computação em nuvem para aplicações de *High Performance Computing* (HPC), oferecendo recursos sob demanda que permitem a execução de cargas de trabalho intensivas em computação e E/S. Entre os serviços disponíveis, destacam-se as instâncias otimizadas para HPC, como as famílias C, H e P, que fornecem alto poder de processamento aliado a interconexões de baixa latência por meio do *Elastic Fabric Adapter* (EFA), permitindo a utilização de bibliotecas como MPI em escala. Para atender aplicações intensivas em dados, a AWS integra sistemas de armazenamento de alto desempenho, como o Amazon FSx for Lustre, que viabiliza acesso paralelo e em larga escala, semelhante a sistemas de arquivos utilizados em supercomputadores tradicionais. Essa combinação de infraestrutura elástica, rede de alto desempenho e suporte a ferramentas amplamente usadas em HPC torna a nuvem da AWS uma alternativa competitiva frente a *clusters* locais, especialmente em cenários que demandam escalabilidade e flexibilidade JACKSON *et al.* (2010); TRAN e THEKKEDATH (2020).

2.5 O Programa Mensal da Operação (PMO)

No setor elétrico brasileiro, o Programa Mensal da Operação (PMO) é um documento técnico elaborado pelo Operador Nacional do Sistema Elétrico (ONS) que estabelece as diretrizes de curto prazo para a operação do Sistema Interligado Nacional (SIN). O PMO contempla projeções de carga, disponibilidade de geração, restrições de transmissão e condições hidrológicas, servindo como referência para a programação da operação hidrotérmica de forma a garantir o equilíbrio entre oferta e demanda de energia elétrica. Além disso, define metas de despacho de usinas hidrelé-

tricas e termelétricas, contribuindo para a segurança energética e para a otimização do uso dos recursos disponíveis. Por sua relevância, o PMO orienta não apenas as decisões operativas do ONS, mas também agentes do setor, incluindo geradores, comercializadores e distribuidores, sendo um instrumento fundamental para a gestão integrada e eficiente do SIN OPERADOR NACIONAL DO SISTEMA ELÉTRICO (2023).

2.6 Algoritmo SDDP

O algoritmo SDDP (*Stochastic Dual Dynamic Programming*), proposto por PEREIRA e PINTO (1991), decompõe o problema de planejamento hidrotérmico em uma malha bidimensional de subproblemas determinísticos: cada subproblema é indexado por um par (estágio, cenário), em que os estágios representam períodos discretos da operação — meses, no caso da operação programada do SIN — e os cenários representam realizações estocásticas das variáveis relevantes, principalmente aflúncias aos reservatórios. A solução do problema é construída em duas fases. Na fase de *convergência*, iterações *forward/backward* constroem progressivamente uma política de operação representada por um conjunto de cortes de Benders, até atingir um critério de convergência. Na fase de *simulação final*, executada após a convergência, a política já fixa é avaliada em larga escala sobre um conjunto amplo de cenários para produzir os resultados operacionais entregues ao planejamento — variáveis primais, multiplicadores duais, custos e despachos. Esta dissertação concentra-se exclusivamente na fase de simulação final.

Nessa fase, os cenários são, por construção, mutuamente independentes: cada cenário corresponde a uma realização estocástica distinta, avaliada sob a mesma política convergida, sem compartilhamento de estado nem coordenação durante a execução. Essa propriedade caracteriza a fase de simulação final do SDDP como uma aplicação do tipo *bag-of-tasks* CIRNE *et al.* (2003): um conjunto de tarefas mutuamente independentes que podem ser distribuídas entre processadores sem sincronização durante a execução. Os cenários são particionados entre os *ranks* MPI e resolvidos em paralelo, de modo que o tempo total da fase de simulação fica limitado pelo cenário mais lento e não pela soma sequencial de todos. A natureza *bag-of-tasks* da fase de simulação é o que torna esta etapa do SDDP escalável em sistemas HPC; é também o que desloca o gargalo da computação para a persistência dos resultados, à medida que o número de cenários cresce.

Após a resolução, os resultados de cada subproblema — variáveis primais, multiplicadores duais e custos — precisam ser persistidos em disco, pois alimentam etapas subsequentes do próprio algoritmo e análises estatísticas posteriores.

A granularidade temporal adotada para os resultados de cada estágio tem im-

pacto direto sobre o volume de dados produzido por iteração. A abordagem tradicional do SDDP agrega a operação dentro de cada estágio em poucos blocos representativos de carga (*patamares*: pesado, médio e leve), reduzindo cada estágio a um pequeno número de pontos operativos por cenário. A operação programada moderna, entretanto, especialmente com a crescente penetração de fontes intermitentes como solar e eólica, requer resolução horária para capturar adequadamente horários de ponta, vales e rampas associadas à variabilidade dessas fontes. Esta dissertação foca exatamente no regime de resultados horários, em que cada estágio de um cenário gera dados proporcionais ao número de horas simuladas — ordem de magnitude maior que o caso tradicional por blocos. É nesse regime que o gargalo de E/S se torna efetivamente dominante: o volume agregado escrito por iteração cresce linearmente com o número de cenários, com o número de estágios e com a granularidade temporal interna de cada estágio, e a estratégia de implementação dessas escritas determina diretamente a escalabilidade da aplicação em ambientes HPC.

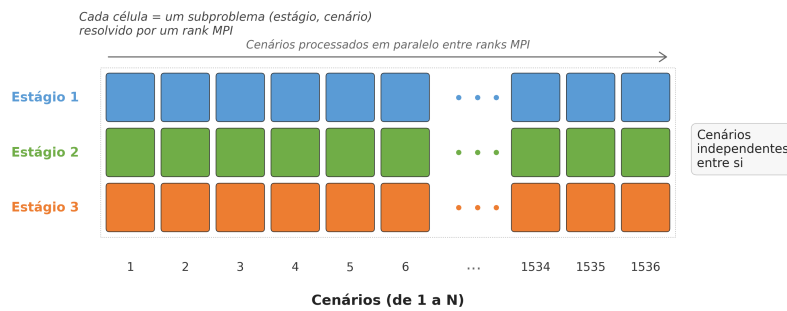


Figura 2.3: Estrutura de cenários e estágios do SDDP. Cada célula corresponde a um subproblema (estágio, cenário). Os cenários são mutuamente independentes e podem ser distribuídos entre *ranks* MPI, caracterizando uma aplicação *bag-of-tasks*.

2.6.1 Padrão dos dados de saída

Como o SDDP organiza a simulação por estágios e cenários, os arquivos de saída também podem ser descritos a partir dessas duas dimensões. No formato mais agregado, os resultados de um par (estágio, cenário) são organizados por patamares de carga internos ao estágio. Nesse caso, a coluna de patamar de carga identifica o intervalo agregado da operação representado em cada linha. A Figura 2.4 mostra um exemplo com cinco patamares. Esse formato reduz o número de linhas por estágio e cenário, mas também reduz a resolução temporal disponível para análise.

estágio	cenário	patamar	A_1	$\dots$	A_N
e_1	c_1	p_1	$v_{1,1,1,1}$	$\dots$	$v_{1,1,1,N}$
e_1	c_1	p_2	$v_{1,1,2,1}$	$\dots$	$v_{1,1,2,N}$
e_1	c_1	p_3	$v_{1,1,3,1}$	$\dots$	$v_{1,1,3,N}$
e_1	c_1	p_4	$v_{1,1,4,1}$	$\dots$	$v_{1,1,4,N}$
e_1	c_1	p_5	$v_{1,1,5,1}$	$\dots$	$v_{1,1,5,N}$

Figura 2.4: Exemplo de arquivo de saída por patamar de carga do SDDP com cinco patamares por estágio e cenário.

Com a inserção da matriz de energias renováveis, esse formato agregado deixa de ser suficiente para representar adequadamente a variabilidade intradiária da geração. Fontes como solar e eólica podem variar significativamente dentro de um mesmo estágio, e essa variação precisa ser preservada nos resultados. Por isso, o formato de saída passa a registrar valores horários. Para uma base mensal com 30 dias, cada par (estágio, cenário) pode gerar 720 linhas, uma para cada hora. Cada linha armazena os identificadores de estágio, cenário e hora, seguidos pelos valores associados aos agentes do sistema, numerados de 1 a N .

A Figura 2.5 ilustra o formato horário. A estrutura explicita a granularidade temporal: para cada estágio e cenário, há uma sequência de linhas que varia de $h = 1$ até $h = 720$, e cada linha contém os valores dos agentes $A_1, \dots, A_N$. O número de agentes tem impacto direto no tamanho de cada bloco de dados. Com 1 agente, um bloco horário de 1 estágio e 1 cenário possui 4 colunas: estágio, cenário, hora e A_1 . Considerando valores de 4 bytes, esse bloco contém $4 \times 720 \times 4 = 11.520$ bytes. Com 300 agentes, o mesmo bloco passa a ter 303 colunas: estágio, cenário, hora e 300 colunas de agentes. Nesse caso, o volume passa para $303 \times 720 \times 4 = 872.640$ bytes por par (estágio, cenário). Portanto, além da granularidade horária, o número de agentes amplia substancialmente o tamanho das mensagens e das escritas enviadas ao sistema de arquivos.

estágio	cenário	hora	A_1	A_2	$\dots$	A_N
e_1	c_1	1	$v_{1,1,1,1}$	$v_{1,1,1,2}$	$\dots$	$v_{1,1,1,N}$
e_1	c_1	2	$v_{1,1,2,1}$	$v_{1,1,2,2}$	$\dots$	$v_{1,1,2,N}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
e_1	c_1	720	$v_{1,1,720,1}$	$v_{1,1,720,2}$	$\dots$	$v_{1,1,720,N}$
e_1	c_2	1	$v_{1,2,1,1}$	$v_{1,2,1,2}$	$\dots$	$v_{1,2,1,N}$

Figura 2.5: Exemplo de arquivo de saída horário do SDDP com 720 horas por estágio e colunas de valores para agentes $A_1, \dots, A_N$.

2.6.2 Arquitetura atual de E/S do SDDP

Na arquitetura atual do SDDP, apenas um *rank* MPI (o *rank* 1) é designado para realizar todas as operações de E/S do sistema. Cada nó de computação, após resolver sua respectiva tarefa, envia os resultados por meio de um *buffer* de dados utilizando a função `MPI_Send`. Esse *buffer* é então recebido pelo processo escritor através de uma chamada `MPI_Recv` e, posteriormente, os dados são gravados em um sistema de arquivos distribuído. A Figura 2.6 ilustra o funcionamento dessa arquitetura.

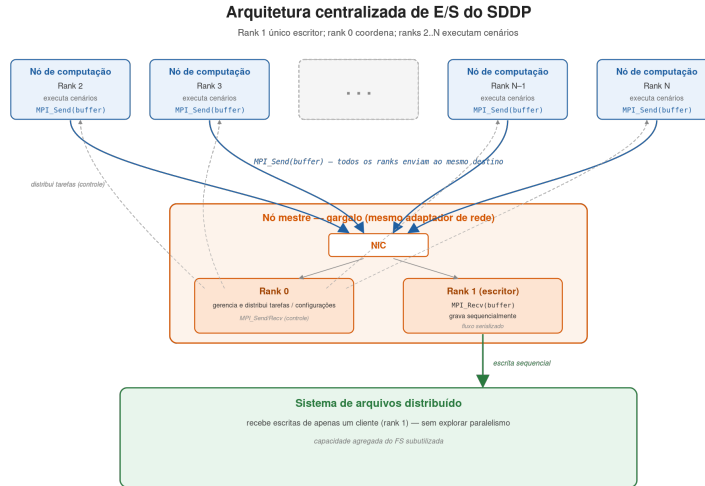


Figura 2.6: Algoritmo SDDP com processo escritor.

Independentemente do número de nós computacionais utilizados, apenas um processo é responsável pelas operações de E/S. Cada nó utiliza sua banda efetiva para enviar os *buffers* de dados, enquanto o processo escritor utiliza sua banda para recebê-los. A primeira hipótese para as limitações de escalabilidade nessa

arquitetura está relacionada ao adaptador de rede do servidor que executa o processo mestre.

Adicionalmente, o *rank* 0 é responsável pelo gerenciamento e controle da aplicação, como a distribuição de tarefas para os demais *ranks* e o envio/recebimento das configurações do sistema. Essas atividades também utilizam o mesmo adaptador de rede empregado na recepção dos *buffers* de dados. Outra hipótese é que o processo escritor opera continuamente em um fluxo sequencial de recepção e escrita dos dados, o que pode representar um gargalo adicional, sobretudo em sistemas com alto volume de dados. Por fim, se houver atrasos na recepção ou na gravação dos *buffers* de dados por parte do processo mestre, os nós computacionais podem ser obrigados a aguardar, gerando sobrecarga no envio dos *buffers* e aumentando o custo computacional da execução paralela.

Capítulo 3

Paralelização dos Mecanismos de E/S do SDDP

Este capítulo apresenta a proposta de paralelização dos mecanismos de entrada e saída (E/S) do SDDP. A solução substitui o modelo atual, por uma organização baseada em MPI-IO sobre o sistema de arquivos Lustre. Nessa abordagem, cada *rank* grava diretamente seus próprios resultados em regiões independentes do espaço de saída, evitando a concentração das operações de E/S em um único processo.

O ponto central da proposta é explorar, simultaneamente, o paralelismo da aplicação MPI e o paralelismo do sistema de arquivos. Como os resultados produzidos por diferentes *ranks* são independentes, suas escritas podem ser posicionadas previamente e executadas em paralelo. No Lustre, os arquivos são distribuídos em *stripes* entre diferentes *Object Storage Targets* (OSTs); assim, múltiplas escritas independentes podem ser atendidas por OSTs distintos, aumentando a largura de banda agregada e reduzindo a contenção observada no modelo com escritor centralizado.

Dessa forma, a proposta visa reduzir dois gargalos da arquitetura original. O primeiro é o gargalo de comunicação, pois os dados deixam de ser enviados ao processo escritor antes de serem persistidos. O segundo é o gargalo de armazenamento, pois a escrita passa a ser distribuída entre os recursos do sistema de arquivos paralelo, em vez de ser serializada por um único *rank*. A Figura 3.2 sintetiza essa nova organização.

3.1 Uso de MPI-IO com Lustre para escalabilidade de E/S

Como hipótese para mitigar essas limitações, propõe-se a substituição do modelo atual de E/S por uma abordagem MPI-IO, baseada na interface MPI-IO em conjunto com o sistema de arquivos distribuído Lustre.

A interface MPI-IO, parte do padrão MPI-2, fornece mecanismos para a realização de operações de E/S por múltiplos processos MPI sobre arquivos compartilhados. Diferentemente do modelo tradicional de comunicação ponto a ponto (MPI_Send/MPI_Recv), o MPI-IO permite que cada processo realize operações diretas de escrita em arquivos, sem a necessidade de coordenação com um processo único de escrita THAKUR *et al.* (1999b). Essa flexibilidade possibilita que os próprios processos responsáveis pela computação escrevam seus resultados diretamente em disco, utilizando *offsets* distintos e evitando conflitos de acesso. Isso é particularmente relevante em aplicações como o SDDP, em que os dados produzidos por cada *rank* são independentes e podem ser gravados em posições previamente determinadas de um arquivo compartilhado, ou mesmo em arquivos separados.

Aliado ao MPI-IO, propõe-se o uso do sistema de arquivos Lustre, amplamente adotado em ambientes de computação de alto desempenho (HPC). O Lustre foi projetado para oferecer alto rendimento e paralelismo no acesso a arquivos, distribuindo os dados entre múltiplos servidores de armazenamento (OSTs — *Object Storage Targets*) BRAAM (2002). Essa característica torna o Lustre particularmente eficiente em cenários onde múltiplos nós executam operações simultâneas de escrita, como se espera com a integração do MPI-IO. A combinação das duas tecnologias permite que os *ranks* MPI realizem operações de E/S em paralelo, aproveitando ao máximo a largura de banda disponível no sistema e reduzindo significativamente os gargalos causados pela concentração da escrita.

Espera-se que essa nova arquitetura traga ganhos substanciais de desempenho. Primeiramente, ao eliminar o processo escritor único, a aplicação poderá distribuir dinamicamente a carga de E/S entre os próprios processos computacionais, reduzindo a sobrecarga sobre o adaptador de rede do nó mestre. Em segundo lugar, a redução da dependência de *buffers* intermediários e de sincronizações entre *ranks* minimiza a latência entre o término da computação e a persistência dos resultados. Além disso, a utilização do paralelismo nativo do Lustre deverá resultar em um melhor aproveitamento dos recursos de armazenamento, tornando o sistema mais escalável à medida que aumenta o número de nós. Por fim, essa abordagem alinha-se às boas práticas adotadas em aplicações científicas de larga escala, promovendo maior portabilidade, eficiência e adaptabilidade a infraestruturas HPC modernas. A Figura 3.2 descreve o funcionamento da proposta.

3.2 Organização dos arquivos compartilhados de resultados

Os resultados gerados pelo SDDP são organizados em arquivos de saída com dimensões associadas a estágio, cenário e hora. A dimensão estágio pode representar meses ou semanas, dependendo da base de dados utilizada, enquanto as dimensões cenário e hora representam, respectivamente, as realizações estocásticas simuladas e a discretização horária dos resultados. Assim, cada registro de saída pode ser identificado por uma tupla (e, c, h) , em que e indica o estágio, c o cenário e h a hora.

Na arquitetura proposta, esses resultados podem ser gravados em arquivos compartilhados por múltiplos processos MPI. A escrita permanece independente: cada *rank* calcula previamente a região do arquivo que corresponde aos resultados sob sua responsabilidade e escreve diretamente nessa posição por meio de chamadas MPI-IO com *offset* explícito, como `MPI_File_write_at` ou sua variante não bloqueante. Com isso, não é necessário encaminhar todos os blocos para um processo escritor central, e também não é necessário sincronizar todos os processos em uma operação coletiva de escrita.

Considerando um arquivo linearizado na ordem estágio–cenário–hora, o *offset* de um registro pode ser calculado a partir de

$$\text{offset}(e, c, h) = \text{base} + (((e \times N_c + c) \times N_h + h) \times S_r), \quad (3.1)$$

em que N_c é o número de cenários, N_h é o número de horas por estágio e S_r é o tamanho, em bytes, de cada registro de resultado. Essa organização permite que cada processo escreva em uma região disjunta do arquivo. Quando um *rank* é responsável por um conjunto contíguo de cenários, horas ou estágios, a escrita pode ser emitida como um bloco contíguo, evitando o uso de *strides* ou de *file views* complexas. Essa é uma vantagem importante para o SDDP, pois reduz a complexidade da descrição do acesso e favorece requisições maiores ao sistema de arquivos.

A escolha por escrita independente, em vez de escrita coletiva, decorre do padrão de execução do SDDP. Os subproblemas associados aos cenários podem terminar em momentos diferentes, e impor uma sincronização global para gravar resultados pode introduzir espera artificial entre processos. Além disso, quando os resultados são produzidos em blocos pequenos, sincronizar todos os *ranks* a cada etapa de escrita pode aumentar a contenção e reduzir a sobreposição entre computação e E/S. A escrita independente com *offsets* explícitos preserva a autonomia dos processos e permite que a aplicação explore uma abordagem assíncrona: os blocos já produzidos podem ser encaminhados ao sistema de arquivos enquanto outros processos

continuam avançando na computação.

No Lustre, o arquivo compartilhado é dividido em *stripes* distribuídos entre diferentes OSTs. Portanto, embora a aplicação enxergue um único arquivo lógico, as regiões escritas por diferentes processos podem ser atendidas por OSTs distintos, dependendo do *stripe size*, do *stripe count* e do *offset* de cada escrita. A Figura 3.1 ilustra essa organização: na parte superior, o arquivo lógico é particionado em regiões de resultados por estágio, cenário e hora; na parte inferior, esses pedaços são distribuídos em *stripes* sobre os OSTs do Lustre.

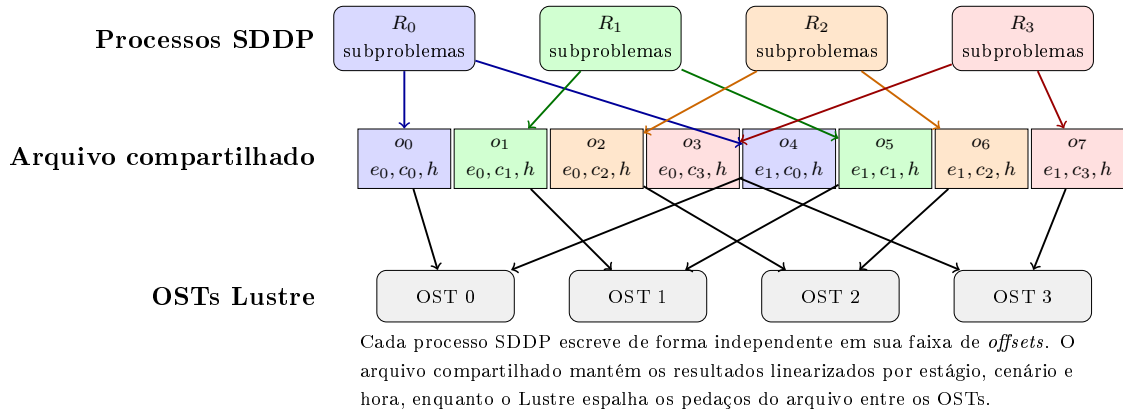


Figura 3.1: Escrita independente dos processos SDDP em um arquivo compartilhado e distribuição dos pedaços em OSTs do Lustre.

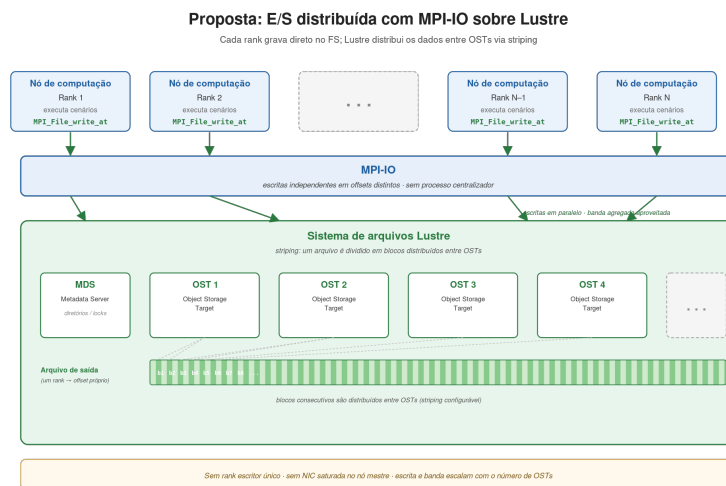


Figura 3.2: Proposta com MPI-IO e Lustre.

Capítulo 4

Avaliações

Este capítulo apresenta a avaliação comparativa entre a implementação atual de entrada e saída (E/S) e a implementação proposta, com o objetivo de investigar ganhos de desempenho e melhorias de escalabilidade, principalmente em ambientes de computação em nuvem. A análise concentra-se em métricas de tempo de execução, tempo de comunicação, largura de banda e tempo de bloqueio das operações de E/S, considerando o impacto das estratégias adotadas para operações paralelas de E/S em cenários de alto desempenho THAKUR *et al.* (1999b); CARNS *et al.* (2009).

A base de dados utilizada nos experimentos corresponde ao PMO/PAR de outubro de 2023. Para a etapa de simulação horária, que constitui o foco principal deste estudo, foram considerados 1.536 cenários em ambos os experimentos (AWS e SDumont), com três estágios mensais.

Como cada *rank* MPI apresentou consumo de memória residente superior a 8 GB, optou-se, em ambos os experimentos, por mapear apenas um *rank* por núcleo físico — isto é, sem o uso do *hyper-threading*. Essa decisão é imposta pela memória total disponível por nó: com 384 GB por nó nos dois ambientes, ativar *hyper-threading* e dobrar o número de *ranks* ultrapassaria a memória física agregada, inviabilizando a execução. O número de *ranks* por nó foi, portanto, fixado no número de núcleos físicos da instância correspondente.

Os experimentos conduzidos neste trabalho caracterizam um cenário de *strong scaling*, no qual o tamanho do problema permanece fixo enquanto o número de nós computacionais é progressivamente aumentado AMDAHL (1967). Nesse contexto, o acréscimo de recursos implica na redução do volume de trabalho por nó, sendo esperado, idealmente, que o tempo total de execução diminua de forma aproximadamente linear com o aumento do paralelismo. No entanto, essa tendência teórica pode não se concretizar na prática devido à presença de gargalos sistêmicos, especialmente aqueles relacionados às operações de entrada e saída (E/S) e à comunicação entre processos. À medida que o número de nós cresce, a intensificação da concorrência por recursos compartilhados — como o sistema de arquivos e a rede —

pode introduzir sobrecargas adicionais, limitando a eficiência do escalonamento e reduzindo os ganhos esperados de desempenho.

O tamanho dos blocos de dados foi considerado um fator determinante para a análise de desempenho de E/S paralela. A Figura 4.1 apresenta a distribuição agregada do tamanho dos blocos de dados observados ao longo do experimento. Observa-se que a distribuição é dominada por blocos de dados pequenos: 1.801.728 blocos, correspondentes a 80,1% do total, possuem tamanho de até 1 KB. As demais faixas concentram 152.082 blocos até 128 KB (6,8%), 267.264 blocos até 1 MB (11,9%) e 27.648 blocos até 50 MB (1,2%). Esse perfil pode comprometer a escalabilidade, já que operações com granularidade fina tendem a aumentar a sobrecarga de metadados e a subutilizar a largura de banda disponível THAKUR *et al.* (1999b); CARNS *et al.* (2009). Adicionalmente, será avaliado o tempo de envio dos blocos de dados agrupados por faixas de tamanho, de forma a analisar o impacto da granularidade das operações no desempenho da comunicação e da E/S.

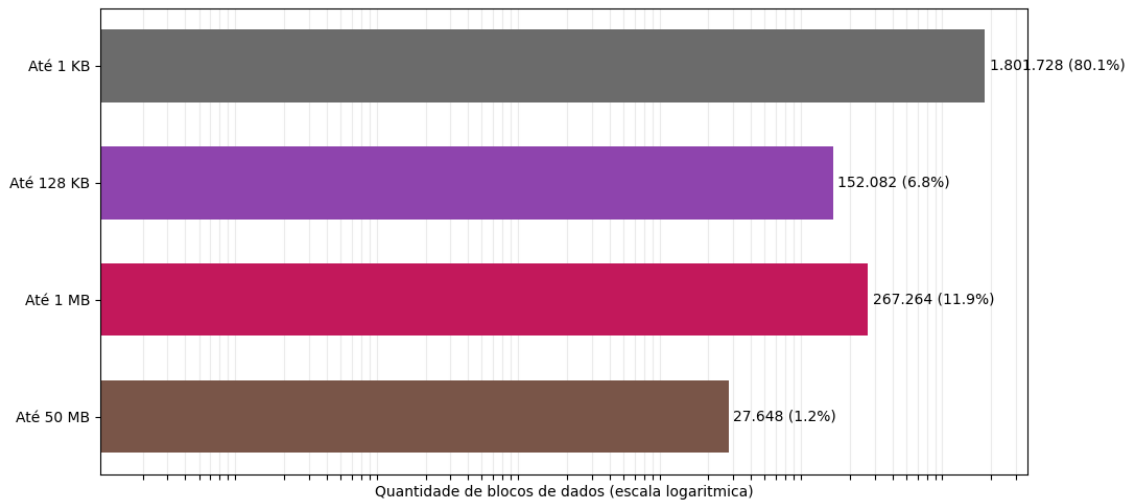


Figura 4.1: Distribuição agregada da quantidade de blocos de dados por faixa de tamanho no experimento, em escala logarítmica.

4.1 Experimento SDumont

O Experimento SDumont corresponde ao supercomputador Santos Dumont (SDumont), adquirido junto à empresa francesa EVIDEN/ATOS/BULL, que está localizado na sede do Laboratório Nacional de Computação Científica (LNCC), em Petrópolis-RJ, atuando como nó central (Tier-0) do Sistema Nacional de Processamento de Alto Desempenho (SINAPAD). Nesse experimento foram utilizados até 32 nós de processamento, cada um equipado com dois processadores Intel Xeon Cascade Lake Gold 6252, totalizando 24 núcleos físicos por nó, com suporte a *hyper-*

threading, o que resulta em 48 processadores lógicos disponíveis. Cada nó conta ainda com 384 GB de memória e interconexão de alta velocidade baseada em Infini-Band EDR de 100 Gb/s, projetada para reduzir a latência e aumentar a eficiência em aplicações paralelas de larga escala. Nos experimentos, foi utilizada a biblioteca MPICH2 versão 3.2 e um sistema de arquivos paralelo Lustre v2.12 implementado através do ClusterStor L300 da Cray/HPE. Essa implementação do Lustre é composta por 1 nó MDS (com 1 MDT) e 6 nós OSSs (cada um servindo 1 OST) e *stripes* de 1 MB, disponibilizando um espaço total de armazenamento de 1,1 PB. Pelos motivos discutidos na introdução do capítulo, também aqui se configurou um *rank* por núcleo físico, totalizando 24 cenários simultâneos por nó.

4.1.1 Avaliação da implementação atual

Esta subseção analisa o desempenho da arquitetura atual de E/S do SDDP no Experimento SDumont. A Tabela 4.1 sumariza os resultados, considerando 1.536 cenários, três estágios mensais e 24 *ranks* por nó.

O tempo total de execução, calculado como a soma do tempo médio de simulação e do tempo médio de comunicação por processo, não apresenta tendência monotônica de redução com o aumento do número de nós: cai de 9.377 s em 2 nós para 6.842 s em 4 nós, permanece próximo desse patamar em 8 nós, com 6.981 s, e volta a crescer em 16 nós, atingindo 9.082 s. A banda agregada cai consistentemente, de 4,57 Gb/s em 2 nós para 0,84 Gb/s em 16 nós. O tempo de comunicação representa cerca de 10–15% do total, enquanto a parcela de E/S, já embutida no tempo de simulação, cresce de 9,54% para 72,70%. Esse comportamento caracteriza um regime degradado sob *strong scaling*, no qual a contenção do processo escritor mais do que compensa os ganhos de paralelismo computacional.

Tabela 4.1: Desempenho do Experimento SDumont com a implementação atual. O tempo total corresponde à soma do tempo médio de simulação e comunicação; o tempo de I/O é uma parcela interna da simulação e não é somado novamente. A eficiência é calculada em relação à base de 2 nós.

Nós	Banda (Gb/s)	I/O (s)	% I/O	Comunicação (s)	Total (s)	Speedup	Eficiência (%)
2	4,57 ± 0,56	894,81 ± 341,58	9,54%	932,59 ± 48,27	9 376,80 ± 349,11	1,00×	100,0%
4	2,04 ± 0,54	2 116,61 ± 482,68	30,94%	921,08 ± 55,50	6 841,99 ± 479,20	1,37×	68,5%
8	1,52 ± 1,01	4 013,54 ± 220,86	57,49%	1 014,67 ± 36,82	6 981,25 ± 163,40	1,34×	33,5%
16	0,84 ± 0,75	6 602,61 ± 671,38	72,70%	1 277,79 ± 30,77	9 082,37 ± 174,91	1,03×	12,9%

4.1.2 Avaliação da implementação MPI-IO com escritas independentes

Esta subseção analisa o desempenho da arquitetura proposta no Experimento SDumont, baseada em escritas independentes com MPI-IO sobre o sistema de arquivos Lustre operado pelo ClusterStor L300. A Tabela 4.2 consolida os resultados.

Na implementação MPI-IO, o tempo total de execução passa de 10.344 s em 2 nós para 6.431 s em 4 nós e 6.343 s em 8 nós, voltando a crescer para 7.593 s em 16 nós. Em comparação com a implementação atual, a versão MPI-IO apresenta sobrecusto em 2 nós, com aumento de aproximadamente 10,3% no tempo total, mas reduz o tempo em 4, 8 e 16 nós, com ganhos aproximados de 6,0%, 9,1% e 16,4%, respectivamente. A banda agregada do *layer* de escrita permanece modesta, oscilando entre 1,6 e 2,8 Gb/s ao longo das escalas avaliadas. Os desvios padrão expressivos a partir de 8 nós, da ordem do próprio valor médio para o tempo de E/S, indicam alta variabilidade e sugerem limitação por metadados, coordenação e concorrência no Lustre.

Tabela 4.2: Desempenho do Experimento SDumont com a implementação MPI-IO. O tempo total corresponde à soma do tempo médio de simulação e comunicação; o tempo de I/O é uma parcela interna da simulação e não é somado novamente. A eficiência é calculada em relação à base de 2 nós.

Nós	Banda (Gb/s)	I/O (s)	% I/O	Comunicação (s)	Total (s)	Speedup	Eficiência (%)
2	1,92 ± 0,22	1 505,19 ± 338,26	14,55%	968,96 ± 134,71	10 343,93 ± 511,69	1,00×	100,0%
4	2,76 ± 0,53	1 186,44 ± 414,20	18,45%	850,55 ± 177,58	6 430,57 ± 489,98	1,61×	80,5%
8	2,08 ± 0,63	1 884,36 ± 2 092,62	29,71%	976,98 ± 213,86	6 342,98 ± 2 424,11	1,63×	40,8%
16	1,63 ± 0,56	1 958,12 ± 2 025,19	25,79%	1 185,57 ± 487,98	7 592,64 ± 2 840,08	1,36×	17,0%

4.1.3 Análise comparativa

Os resultados das duas subseções anteriores são consolidados visualmente nas figuras a seguir, que apresentam, lado a lado, os perfis de banda agregada, tempo de execução e tempo de bloqueio de envio em função do número de nós para as duas implementações no Experimento SDumont.

A Tabela 4.3 resume os principais indicadores comparativos do Experimento SDumont. A coluna de melhoria no tempo total compara diretamente a implementação MPI-IO com a implementação atual para o mesmo número de nós, calculada como $(Total_{atual} - Total_{MPIIO})/Total_{atual}$. Valores positivos indicam redução do tempo total, enquanto valores negativos indicam piora. Embora a MPI-IO apresente sobrecusto em 2 nós, ela passa a reduzir o tempo total a partir de 4 nós,

chegando a 16,4% de melhoria em 16 nós. O ganho de banda também cresce com a escala, alcançando 1,94× em 16 nós, enquanto a redução do tempo de E/S chega a 70,3% nessa configuração.

Tabela 4.3: Resumo comparativo entre a implementação atual e MPI-IO no Experimento SDumont. A melhoria no total e a melhoria de I/O são calculadas em relação à implementação atual; valores negativos indicam piora.

Nós	Total atual (s)	Total MPI-IO (s)	Melhoria no total (%)	Banda atual (Gb/s)	Banda MPI-IO (Gb/s)	Ganho de banda	I/O atual (s)	I/O MPI-IO (s)	Melhoria I/O (%)
2	9 376,80	10 343,93	−10,3%	4,57	1,92	0,42×	894,81	1 505,19	−68,2%
4	6 841,99	6 430,57	6,0%	2,04	2,76	1,36×	2 116,61	1 186,44	43,9%
8	6 981,25	6 342,98	9,1%	1,52	2,08	1,37×	4 013,54	1 884,36	53,1%
16	9 082,37	7 592,64	16,4%	0,84	1,63	1,94×	6 602,61	1 958,12	70,3%

A Figura 4.2 compara a banda agregada média obtida pela implementação atual e pela implementação MPI-IO. Em 2 nós, a implementação atual apresenta a maior banda observada, com cerca de 4,6 Gb/s, enquanto a versão MPI-IO atinge aproximadamente 1,9 Gb/s. A partir de 4 nós, entretanto, a implementação MPI-IO passa a apresentar maior banda agregada que a implementação atual, alcançando cerca de 2,8 Gb/s em 4 nós, 2,1 Gb/s em 8 nós e 1,6 Gb/s em 16 nós. Esses valores correspondem a ganhos aproximados de 1,36×, 1,37× e 1,94× em relação à implementação atual, respectivamente.

Esse comportamento indica que, em pequena escala, o custo adicional da abordagem MPI-IO ainda não é compensado pelo paralelismo de escrita. Com o aumento do número de nós, a contenção sobre o processo escritor da implementação atual torna-se dominante, levando à queda progressiva da banda agregada. A implementação MPI-IO reduz esse gargalo ao distribuir as operações de E/S, mas também apresenta queda de desempenho nas maiores configurações, indicando que a escalabilidade no SDumont é limitada por fatores adicionais da infraestrutura de armazenamento — detalhados a seguir na decomposição por categoria de bloco (Figura 4.4).

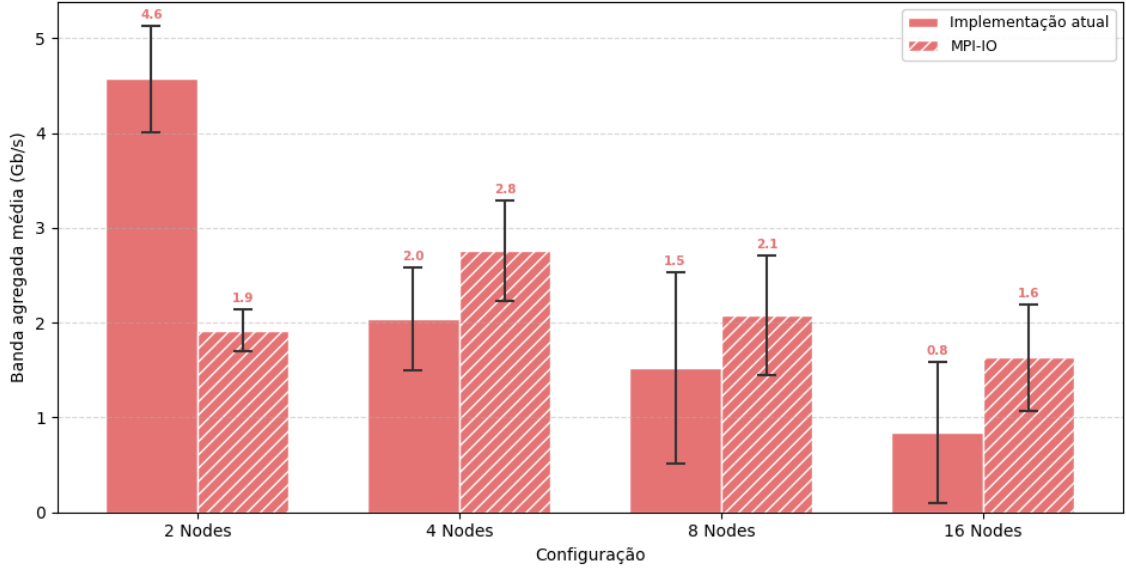


Figura 4.2: Banda agregada média das implementações atual e MPI-IO no Experimento SDumont.

A Figura 4.3 detalha o tempo médio por processo no Experimento SDumont, separando as parcelas de computação, comunicação, E/S e Coordenação MPI-IO. A parcela Coordenação MPI-IO corresponde às chamadas `MPI_File_Open/MPI_File_Close`, que ocorrem *uma única vez* por execução — na abertura e no fechamento do arquivo de saída — e são, portanto, custos de natureza $O(1)$ em relação ao número de cenários processados. As barras de erro indicam somente o desvio padrão do tempo de execução total. Em 2 nós, a implementação MPI-IO apresenta tempo total superior ao da implementação atual, pois o custo dessas operações de coordenação e da própria camada MPI-IO ainda pesa de forma significativa. Nas configurações com 4, 8 e 16 nós, a versão MPI-IO reduz o tempo total em relação à implementação atual, com ganhos mais evidentes nas maiores escalas.

Na implementação atual, a parcela de E/S cresce fortemente com o aumento do número de nós, chegando a representar a maior parte do tempo total em 16 nós. Esse comportamento é compatível com a saturação do processo escritor centralizado. Na implementação MPI-IO, o tempo de E/S é redistribuído entre os processos e permanece mais controlado, embora o custo da Coordenação MPI-IO passe a compor uma fração visível do tempo total.

É importante destacar que a fração relativa da Coordenação MPI-IO observada nestes experimentos é maior do que se espera em execuções reais do SDDP. Como o custo de `MPI_File_Open/MPI_File_Close` é fixo por execução, sua participação no tempo total decresce à medida que a carga computacional cresce: nas execuções aqui realizadas, com apenas três estágios mensais e 1536 cenários, o tempo total

de simulação é relativamente baixo, o que infla o peso percentual dessa parcela. Em execuções de operação programada típicas do SIN, com horizonte de vários anos em resolução mensal e número de cenários muito maior, o custo absoluto de coordenação permanece o mesmo enquanto o tempo total de computação cresce em ordem de magnitude — diluindo a fração relativa da Coordenação MPI-IO para níveis desprezíveis. Assim, a figura evidencia que a abordagem proposta reduz o gargalo centralizado de escrita; o custo residual de coordenação, embora visível na escala dos experimentos, não é uma limitação intrínseca da arquitetura proposta.

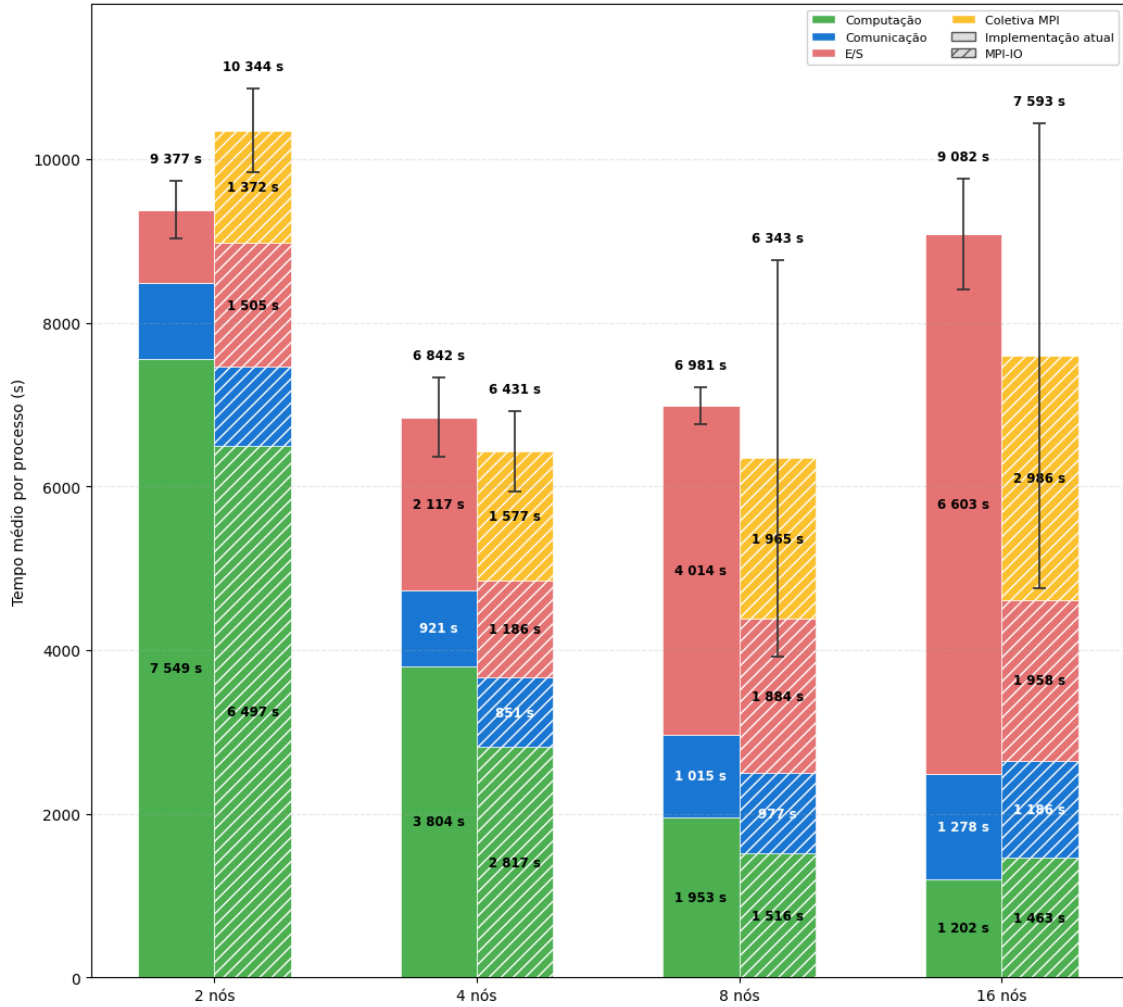


Figura 4.3: Decomposição do tempo médio por processo das implementações atual e MPI-IO no Experimento SDumont.

A Figura 4.4 apresenta o tempo médio de bloqueio das operações de envio em função do tamanho dos blocos de dados no Experimento SDumont, em escala logarítmica, organizada em quatro painéis — um por categoria de tamanho.

O painel (a), correspondente a blocos de até 1 KB, evidencia o principal gargalo adicional do Experimento SDumont. Enquanto a implementação atual mantém o

tempo de envio na ordem de microssegundos — pois a operação reduz-se a um `MPI_Send` para o *rank* escritor —, a implementação MPI-IO passa para a ordem de centenas de milissegundos a alguns segundos por chamada, com o tempo crescendo monotonicamente com o número de nós.

Esse padrão é compatível com o regime de saturação do caminho de dados do Lustre quando muitos clientes emitem escritas pequenas concorrentes sobre um mesmo arquivo já aberto. O efeito não pode ser atribuído a operações de criação, abertura ou fechamento, pois essas ocorrem uma única vez por arquivo, fora do laço de escritas.

Nos painéis (b)–(d), referentes a blocos médios e grandes, o comportamento se inverte: o custo fixo da camada MPI-IO é amortizado pelo volume transferido, e a implementação MPI-IO apresenta tempos de bloqueio significativamente menores que a implementação atual, que sofre com a saturação do processo escritor único. Em 16 nós, por exemplo, a categoria “até 1 MB” reduz o tempo médio de envio de aproximadamente 16 s na implementação atual para cerca de 0,2 s na MPI-IO — redução de aproximadamente duas ordens de grandeza.

Em conjunto, os painéis evidenciam um ponto de troca: para blocos pequenos, o custo fixo por chamada e a contenção no caminho de dados do Lustre dominam e penalizam a abordagem MPI-IO; para blocos médios e grandes, a escrita paralela reduz substancialmente a contenção do escritor único. Esse comportamento — penalidade severa apenas em blocos pequenos, ganho preservado em blocos maiores — também contribui para explicar a queda de banda agregada da Figura 4.2 nas maiores escalas, em que a fração de chamadas pequenas concorrentes sobre o sistema de arquivos cresce com o número de *ranks*.

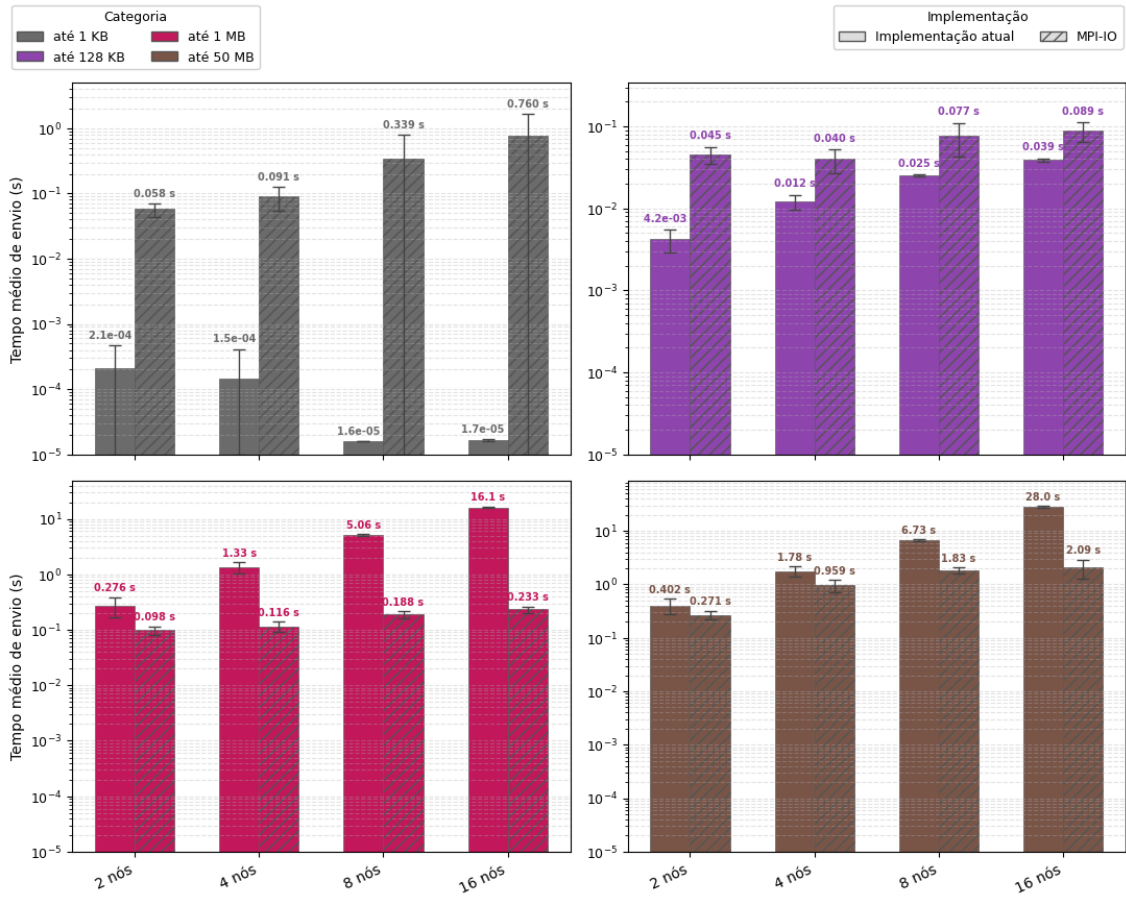


Figura 4.4: Tempo médio de bloqueio das operações de envio por tamanho de bloco de dados nas implementações atual e MPI-IO no Experimento SDumont (escala logarítmica).

Os resultados apresentados nesta seção demonstram que a adoção da abordagem MPI-IO com escritas independentes reduz o gargalo da arquitetura original associado à concentração das operações de E/S no processo escritor. No SDumont, essa redução se reflete em maior banda agregada a partir de 4 nós e em menor tempo total nas configurações de 4, 8 e 16 nós. Entretanto, os desvios observados e a queda de desempenho nas maiores escalas indicam que a infraestrutura de armazenamento ainda impõe limitações por metadados, coordenação e concorrência entre processos. Assim, a estratégia proposta melhora a escalabilidade em relação ao modelo centralizado, mas seu benefício depende das características do sistema de arquivos e da escala de execução.

4.2 Experimento AWS

O Experimento AWS foi configurado na nuvem da Amazon Web Services (AWS), utilizando 32 nós da instância do tipo r7i.12xlarge. Essa instância é baseada na quarta geração de processadores Intel Xeon Scalable (Sapphire Rapids 8488C), operando a 3,2 GHz, e disponibiliza 24 núcleos físicos com suporte a *hyper-threading*, totalizando 48 processadores lógicos. O sistema conta ainda com 384 GB de memória DDR5, que oferece maior largura de banda em relação às gerações anteriores, e um adaptador de rede Ethernet com capacidade de até 18,75 Gbps, projetado para cargas de trabalho intensivas em comunicação. Essa configuração enquadra-se na categoria de instâncias otimizadas para memória da AWS (*memory-optimized*), adequada a aplicações que combinam elevada demanda de processamento paralelo com grandes volumes de dados em memória — perfil característico do SDDP, em que cada *rank* MPI chega a consumir mais de 8 GB de memória residente. Nos experimentos, utilizou-se a biblioteca MPICH2 versão 3.2 e um sistema de arquivos paralelo Lustre versão 2.12 de 12 TB, com 1 MDS de 350 GB e 10 OSTs de 1,165 TB e *stripes* de 1 MB, disponibilizado por meio do serviço Amazon FSx for Lustre. Pelos motivos discutidos na introdução do capítulo, configurou-se um *rank* por núcleo físico, totalizando 24 cenários simultâneos por nó INTEL CORPORATION (2023); AMAZON WEB SERVICES (2024).

4.2.1 Avaliação da implementação atual

Esta subseção analisa o desempenho da arquitetura atual de E/S do SDDP no Experimento AWS. A Tabela 4.4 sumariza os resultados obtidos com a base de dados PMO/PAR de outubro de 2023, considerando 1.536 cenários, três estágios mensais e 24 *ranks* por nó.

Observa-se que o tempo total de execução cai com o aumento do número de nós até 16 nós, passando de aproximadamente 8 971 s em 2 nós para 1 874 s em 16 nós, mas volta a crescer para 2 013 s em 32 nós — caracterizando o regime saturado típico de um cenário de *strong scaling* limitado pela E/S. A banda agregada degrada-se de 60,5 Gb/s em 2 nós para 3,4 Gb/s em 32 nós (mais de uma ordem de grandeza), e o percentual do tempo dedicado à comunicação cresce de cerca de 2% em 2 nós para 15,1% em 32 nós. Esses números são coerentes com o diagnóstico de que o adaptador de rede do nó mestre se torna ponto de contenção à medida que aumenta o número de processos emissores concorrentes.

Na Tabela 4.4, o tempo total corresponde à soma entre o tempo médio de simulação e o tempo médio de comunicação. O tempo de I/O é apresentado como uma parcela interna do tempo de simulação e, por isso, não é somado novamente ao total.

Tabela 4.4: Desempenho do Experimento AWS com a implementação atual, incluindo tempos de I/O, comunicação, tempo total, *speedup*, eficiência e percentual de I/O. O total corresponde ao tempo de simulação mais comunicação, e a eficiência é calculada em relação à base de 2 nós.

Nós	Banda (Gb/s)	I/O (s)	% I/O	Comunicação (s)	Total (s)	Speedup	Eficiência (%)
2	60,51 ± 5,20	30,85 ± 13,23	0,34%	177,51 ± 29,73	8 971,23 ± 323,48	1,00×	100,0%
4	51,28 ± 2,82	78,23 ± 27,79	1,62%	212,76 ± 19,33	4 823,20 ± 40,39	1,86×	93,0%
8	18,94 ± 3,76	201,67 ± 53,32	7,64%	174,04 ± 12,90	2 639,18 ± 32,59	3,40×	85,0%
16	4,52 ± 0,33	527,94 ± 53,07	28,18%	167,77 ± 7,07	1 873,56 ± 52,28	4,79×	59,9%
32	3,37 ± 0,10	996,57 ± 166,98	49,51%	304,96 ± 3,83	2 013,06 ± 25,99	4,46×	27,9%

4.2.2 Avaliação da implementação MPI-IO com escritas independentes

Esta subseção analisa o desempenho da arquitetura proposta, baseada em escritas independentes com MPI-IO sobre o sistema de arquivos Lustre, no Experimento AWS. Nesta abordagem cada processo MPI escreve seus próprios dados diretamente no FSx for Lustre, eliminando o ponto de contenção associado a um único *rank* escritor e explorando o paralelismo dos múltiplos OSTs.

A Tabela 4.5 consolida os resultados obtidos com a implementação MPI-IO no Experimento AWS. Assim como na tabela anterior, o tempo total corresponde à soma entre simulação e comunicação. O tempo de I/O e o tempo de Coordenação MPI-IO, correspondente às chamadas `MPI_File_Open`/`MPI_File_Close`, são parcelas internas do tempo de simulação e não são somados novamente ao total.

Tabela 4.5: Desempenho do Experimento AWS com a implementação MPI-IO, incluindo tempos de I/O, comunicação, tempo total, *speedup*, eficiência e percentual de I/O. O total corresponde ao tempo de simulação mais comunicação, e a eficiência é calculada em relação à base de 2 nós.

Nós	Banda (Gb/s)	I/O (s)	% I/O	Comunicação (s)	Total (s)	Speedup	Eficiência (%)
2	33,80 ± 3,21	71,89 ± 68,92	0,81%	170,21 ± 17,52	8 847,89 ± 127,16	1,00×	100,0%
4	52,22 ± 9,25	31,30 ± 6,94	0,65%	188,12 ± 14,94	4 795,86 ± 18,74	1,84×	92,0%
8	83,49 ± 12,71	26,33 ± 5,28	1,02%	206,21 ± 18,68	2 574,70 ± 31,83	3,44×	86,0%
16	96,17 ± 8,31	20,37 ± 3,98	1,26%	215,97 ± 33,65	1 613,30 ± 45,80	5,48×	68,5%
32	142,98 ± 21,95	18,18 ± 3,54	1,32%	269,76 ± 18,83	1 374,10 ± 61,95	6,44×	40,3%

4.2.3 Análise comparativa

Os resultados das duas subseções anteriores são consolidados visualmente nas figuras a seguir, que apresentam, lado a lado, os perfis de banda agregada, tempo de execução e tempo de bloqueio de envio em função do número de nós para as duas implementações.

A Tabela 4.6 resume os principais indicadores comparativos do Experimento AWS. A melhoria no tempo total é positiva em todas as escalas avaliadas, mas torna-se mais expressiva quando a implementação atual passa a ser dominada pela contenção de E/S: a redução do tempo total cresce de 2,4% em 8 nós para 13,9% em 16 nós e 31,7% em 32 nós. A comparação de banda evidencia o principal ganho da arquitetura proposta: a MPI-IO passa de desempenho similar em 4 nós para $4,41\times$ a banda da implementação atual em 8 nós, $21,27\times$ em 16 nós e $42,49\times$ em 32 nós. A redução do tempo de E/S também cresce com a escala, chegando a 98,2% em 32 nós.

Tabela 4.6: Resumo comparativo entre a implementação atual e MPI-IO no Experimento AWS. A melhoria no total e a melhoria de I/O são calculadas em relação à implementação atual; valores negativos indicam piora.

Nós	Total atual (s)	Total MPI-IO (s)	Melhoria no total (%)	Banda atual (Gb/s)	Banda MPI-IO (Gb/s)	Ganho de banda	I/O atual (s)	I/O MPI-IO (s)	Melhoria I/O (%)
2	8971,23	8847,89	1,4%	60,51	33,80	0,56×	30,85	71,89	-133,0%
4	4823,20	4795,86	0,6%	51,28	52,22	1,02×	78,23	31,30	60,0%
8	2639,18	2574,70	2,4%	18,94	83,49	4,41×	201,67	26,33	86,9%
16	1873,56	1613,30	13,9%	4,52	96,17	21,27×	527,94	20,37	96,1%
32	2013,06	1374,10	31,7%	3,37	142,98	42,49×	996,57	18,18	98,2%

A Figura 4.5 apresenta a evolução da largura de banda agregada média no Experimento AWS. As duas curvas exibem comportamentos qualitativamente opostos: a implementação atual parte de aproximadamente 60,5 Gb/s em 2 nós e degrada-se de forma acentuada com o aumento do paralelismo, atingindo 18,9 Gb/s em 8 nós, 4,5 Gb/s em 16 nós e 3,4 Gb/s em 32 nós — uma redução de mais de uma ordem de grandeza. Em contraste, a implementação MPI-IO parte de 33,8 Gb/s em 2 nós e cresce monotonicamente, alcançando 52,2 Gb/s em 4 nós, 83,5 Gb/s em 8 nós, 96,2 Gb/s em 16 nós e 143,0 Gb/s em 32 nós.

Em 2 nós, a banda agregada da implementação atual é cerca de 79% maior que a da implementação MPI-IO. Esse comportamento é coerente com o fato de que, em pequena escala, o gargalo do processo escritor único ainda não se manifesta, e o custo fixo da coordenação da E/S paralela tem maior peso relativo. A partir de 4 nós, entretanto, a implementação MPI-IO já supera a atual. Em 8 nós, a banda agregada é aproximadamente $4,4\times$ maior; em 16 nós, cerca de $21,3\times$ maior; e, em

32 nós, aproximadamente $42,5\times$ maior. Esse resultado indica que a arquitetura proposta explora a infraestrutura de armazenamento em uma escala completamente diferente da arquitetura baseada em escritor único.

Essa diferença de comportamento decorre diretamente da eliminação do processo escritor único. Na arquitetura atual, todas as escritas convergem para o adaptador de rede de um único nó, que rapidamente se torna ponto de contenção e limita o *throughput* global — explicando a queda contínua da banda observada. Na abordagem MPI-IO, as operações de escrita são distribuídas entre os processos MPI, permitindo que múltiplos fluxos de dados sejam enviados simultaneamente ao sistema de arquivos Lustre, explorando o paralelismo proporcionado pelos múltiplos OSTs. Como consequência, a banda agregada cresce de forma consistente com o número de nós, demonstrando que o sistema de armazenamento ainda não atingiu seu limite nominal na maior configuração avaliada. Dessa forma, a figura evidencia que a adoção de MPI-IO não apenas elimina o gargalo da arquitetura atual, mas também permite um uso significativamente mais eficiente da largura de banda disponível, tornando a aplicação adequada para execuções em larga escala.

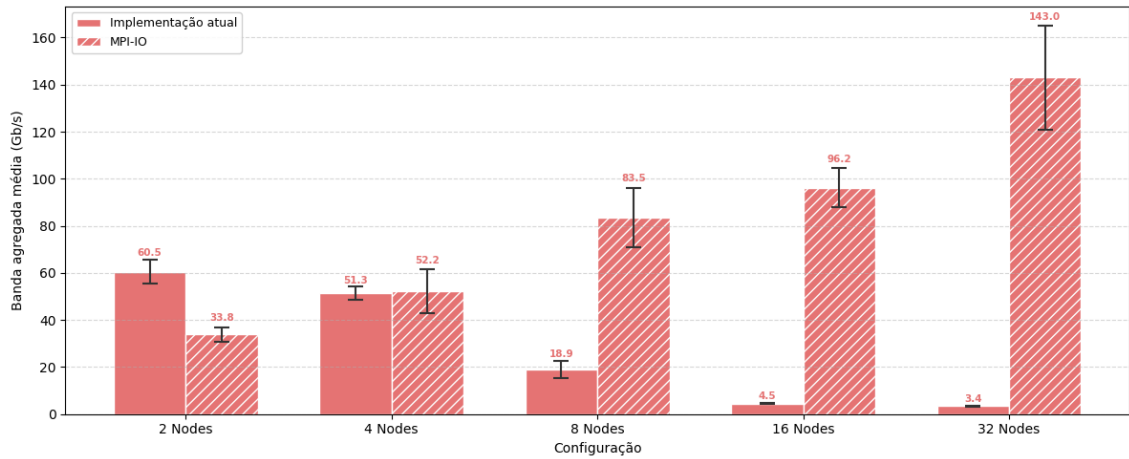


Figura 4.5: Banda agregada média no Experimento AWS.

A Figura 4.6 apresenta a evolução do tempo total de execução no Experimento AWS, contrastando a implementação atual e a implementação MPI-IO sobre o Lustre. As barras de erro representam apenas o desvio padrão do tempo de execução total, e não desvios individuais das parcelas internas. Nas configurações de menor escala, em que a E/S não é o componente dominante, as duas curvas evoluem de forma muito próxima: em 2 nós, $8,97 \times 10^3$ s no modelo atual contra $8,85 \times 10^3$ s no modelo MPI-IO; em 4 nós, $4,82 \times 10^3$ s contra $4,80 \times 10^3$ s. Esse comportamento é coerente, uma vez que, em pequena escala, o tempo de execução é dominado pela computação e a contenção sobre o processo escritor ainda não se manifesta.

A divergência entre as duas curvas torna-se evidente a partir de 8 nós e se acentua de forma marcante nas configurações de maior porte. Enquanto a implementação atual apresenta um regime de saturação — com o tempo de execução passando de $2,64 \times 10^3$ s em 8 nós para $1,87 \times 10^3$ s em 16 nós e voltando a crescer para $2,01 \times 10^3$ s em 32 nós —, a implementação MPI-IO mantém uma redução consistente do tempo total, atingindo $2,57 \times 10^3$ s em 8 nós, $1,61 \times 10^3$ s em 16 nós e $1,37 \times 10^3$ s em 32 nós. Em termos relativos, isso corresponde a uma redução do tempo total de aproximadamente 2% em 8 nós, 14% em 16 nós e 32% em 32 nós em relação ao modelo atual.

A figura também evidencia que, na implementação atual, o crescimento do número de nós deixa de produzir ganho de desempenho a partir de 16 nós e passa a ser, na prática, contraproducente em 32 nós — comportamento característico de regime degradado sob *strong scaling*, no qual a contenção do processo escritor mais do que compensa os ganhos de paralelismo computacional. Em contraste, a curva MPI-IO permanece monotonicamente decrescente em todo o intervalo avaliado, indicando que a infraestrutura de armazenamento ainda não atingiu sua capacidade limite mesmo na maior configuração testada.

Cabe observar que, na implementação MPI-IO, o tempo de E/S torna-se uma fração pequena do tempo total em todas as escalas, permanecendo próximo de 1% a partir de 8 nós. Na implementação atual, por outro lado, a fração de E/S cresce de 7,6% em 8 nós para 28,2% em 16 nós e 49,5% em 32 nós. Na implementação MPI-IO, parte do tempo de simulação passa a ser ocupada pela Coordenação MPI-IO, medida pelas chamadas `MPI_File_Open/MPI_File_Close`: essa parcela cresce de 0,5% em 2 nós para 1,0% em 4 nós, 2,8% em 8 nós, 6,0% em 16 nós e 12,1% em 32 nós. Esse crescimento percentual, contudo, reflete sobretudo a redução do tempo de simulação com o aumento do paralelismo e não um aumento do custo absoluto de coordenação: como `MPI_File_Open/MPI_File_Close` são executadas uma única vez por execução — no início e no final —, seu custo é fixo em relação ao número de cenários processados, e sua fração relativa diminui em execuções com maior carga computacional. Os experimentos aqui apresentados consideram apenas três estágios mensais, o que mantém o tempo total de simulação relativamente curto e infla a fração percentual da Coordenação MPI-IO; em execuções de operação programada típicas do SIN, com horizonte de vários anos e número de cenários muito maior, o custo absoluto dessas chamadas permanece o mesmo enquanto o tempo total cresce em ordem de magnitude, diluindo a parcela de coordenação para níveis desprezíveis. Em todas as escalas avaliadas, e mesmo sob essa inflação relativa do custo de coordenação, a implementação MPI-IO já apresenta tempo total inferior ao da implementação atual.

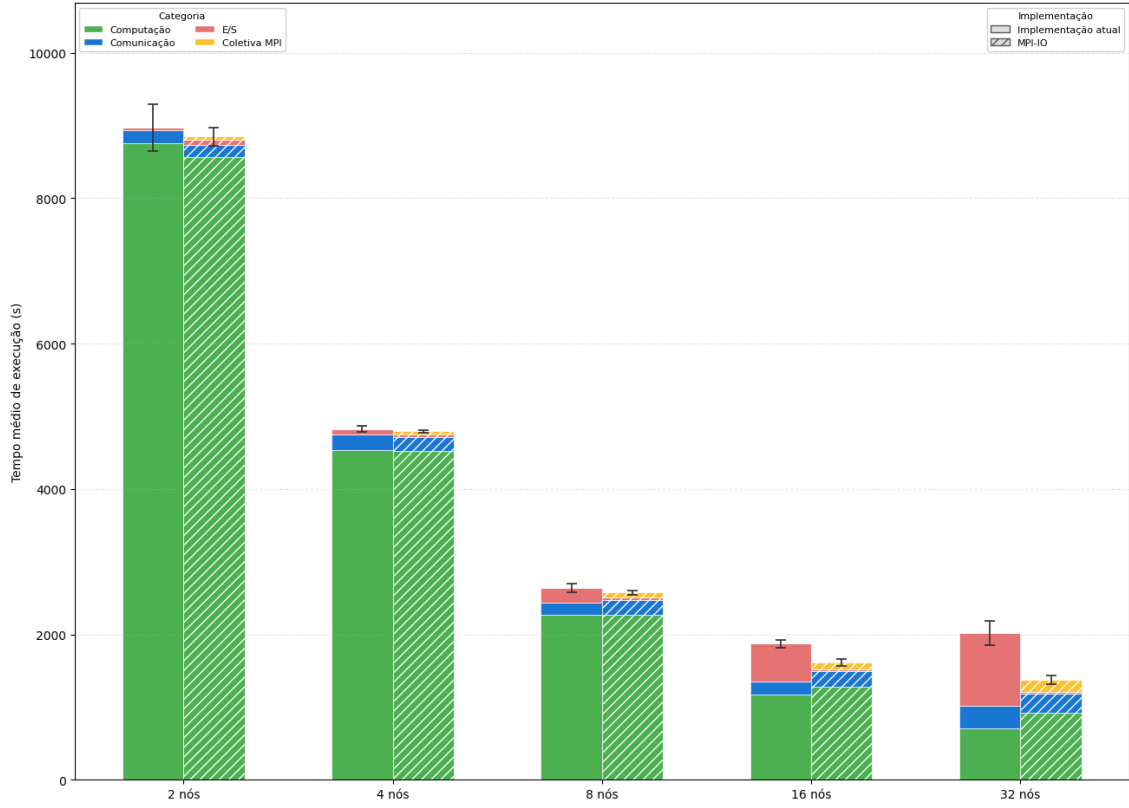


Figura 4.6: Tempo de execução no Experimento AWS.

A Figura 4.7 apresenta o tempo médio de bloqueio das operações de E/S, em segundos, em função do tamanho dos blocos de dados no Experimento AWS, em escala logarítmica. A comparação mostra que, para blocos de dados muito pequenos, a implementação atual apresenta tempos menores, pois o custo fixo das chamadas MPI-IO se torna dominante em relação ao volume transferido. Esse efeito, contudo, ocorre em uma faixa de tempo baixa, da ordem de microssegundos a poucos milissegundos, e não determina o tempo total da aplicação.

Para blocos de dados médios e grandes, que concentram o impacto de E/S nas maiores escalas, o comportamento se inverte. Em 16 e 32 nós, a implementação atual passa a apresentar tempos de bloqueio elevados nas classes de maior tamanho, chegando à ordem de segundos, enquanto a implementação MPI-IO permanece na ordem de centésimos ou décimos de segundo. Assim, a figura reforça o diagnóstico das Figuras 4.5 e 4.6: a abordagem com escritor único é competitiva apenas em pequena escala ou para blocos de dados muito pequenos, mas perde escalabilidade quando o volume de dados e a concorrência entre processos aumentam.

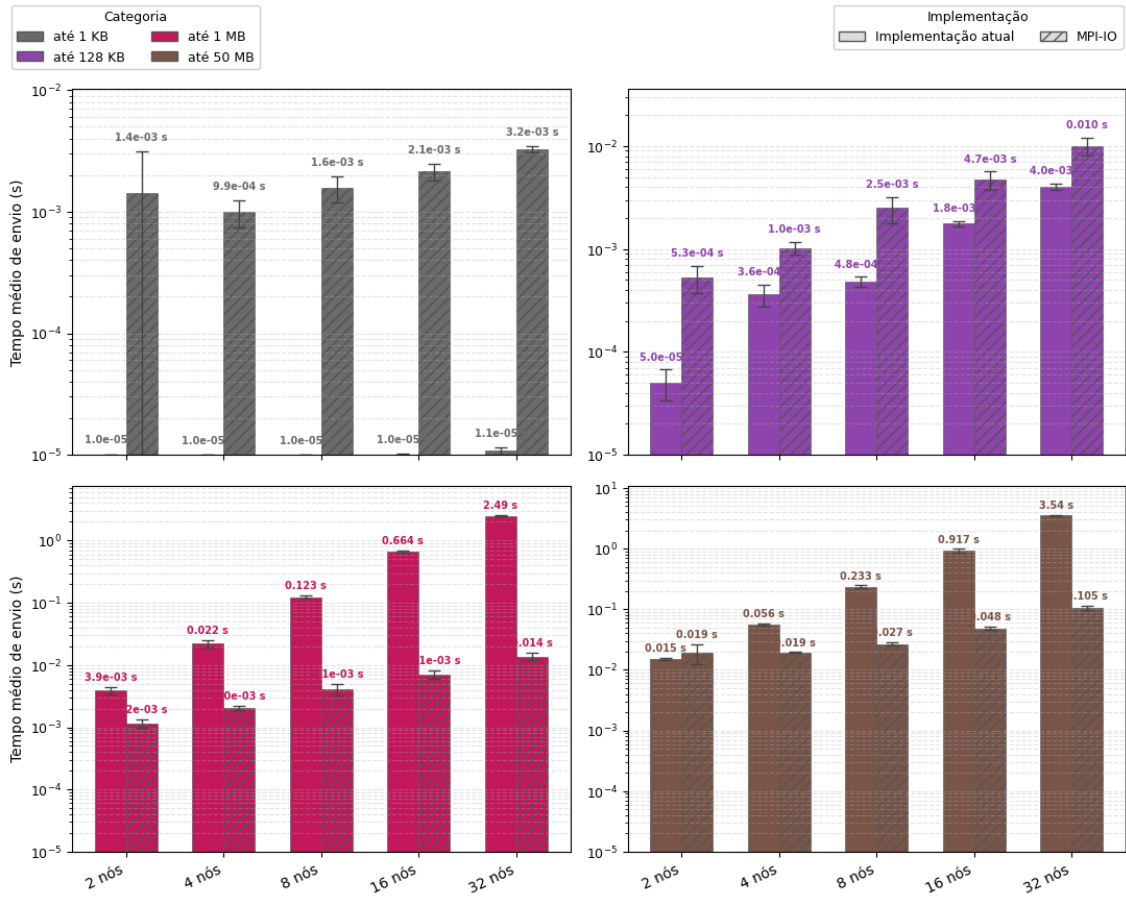


Figura 4.7: Tempo médio de bloqueio das operações de envio em função do tamanho dos blocos de dados no Experimento AWS (escala logarítmica).

4.2.4 Avaliação do impacto do número de OSTs

Esta subseção tem como objetivo avaliar, no Experimento AWS, o impacto do número de *Object Storage Targets* (OSTs) no desempenho das operações de E/S no sistema de arquivos Lustre. Em ambientes HPC, a quantidade de OSTs está diretamente relacionada ao grau de paralelismo de E/S disponível, uma vez que os dados podem ser distribuídos entre múltiplos dispositivos de armazenamento, permitindo acessos concorrentes por diferentes processos.

Para esta avaliação, foram realizados experimentos com a abordagem MPI-IO variando-se a quantidade de OSTs disponíveis no FSx for Lustre, comparando configurações com 4 OSTs e 10 OSTs para execuções com 16 e 32 nós de cálculo. As demais características do experimento foram mantidas constantes, incluindo o volume de dados e o padrão de acesso, de modo a isolar o efeito da infraestrutura de armazenamento.

A Figura 4.8 apresenta o tempo médio de envio por classe de tamanho das

mensagens. Cada subfigura corresponde a uma classe de tamanho, enquanto as barras comparam as configurações MPI-IO com 4 e 10 OSTs para 16 e 32 nós de cálculo. Em 16 nós, o uso de 10 OSTs reduz o tempo médio de envio nas quatro classes analisadas: de 2,6 ms para 2,1 ms nas mensagens de até 1 KB, de 18 ms para 4,7 ms nas mensagens de até 128 KB, de 20 ms para 7,1 ms nas mensagens de até 1 MB e de 87 ms para 48 ms nas mensagens de até 50 MB.

O efeito é mais pronunciado em 32 nós, quando a maior concorrência entre processos amplia a diferença entre as duas configurações. Nesse cenário, os tempos caem de 10 ms para 3,2 ms nas mensagens de até 1 KB, de 111 ms para 10 ms nas mensagens de até 128 KB, de 90 ms para 14 ms nas mensagens de até 1 MB e de 304 ms para 105 ms nas mensagens de até 50 MB. Esses resultados mostram que a redução do número de OSTs aumenta a contenção no acesso aos dados, sobretudo quando o número de nós de cálculo cresce. Embora a escrita seja realizada em paralelo por meio de MPI-IO, a disponibilidade de menos alvos de armazenamento limita o paralelismo efetivo do Lustre e aumenta o tempo observado nas operações de envio.

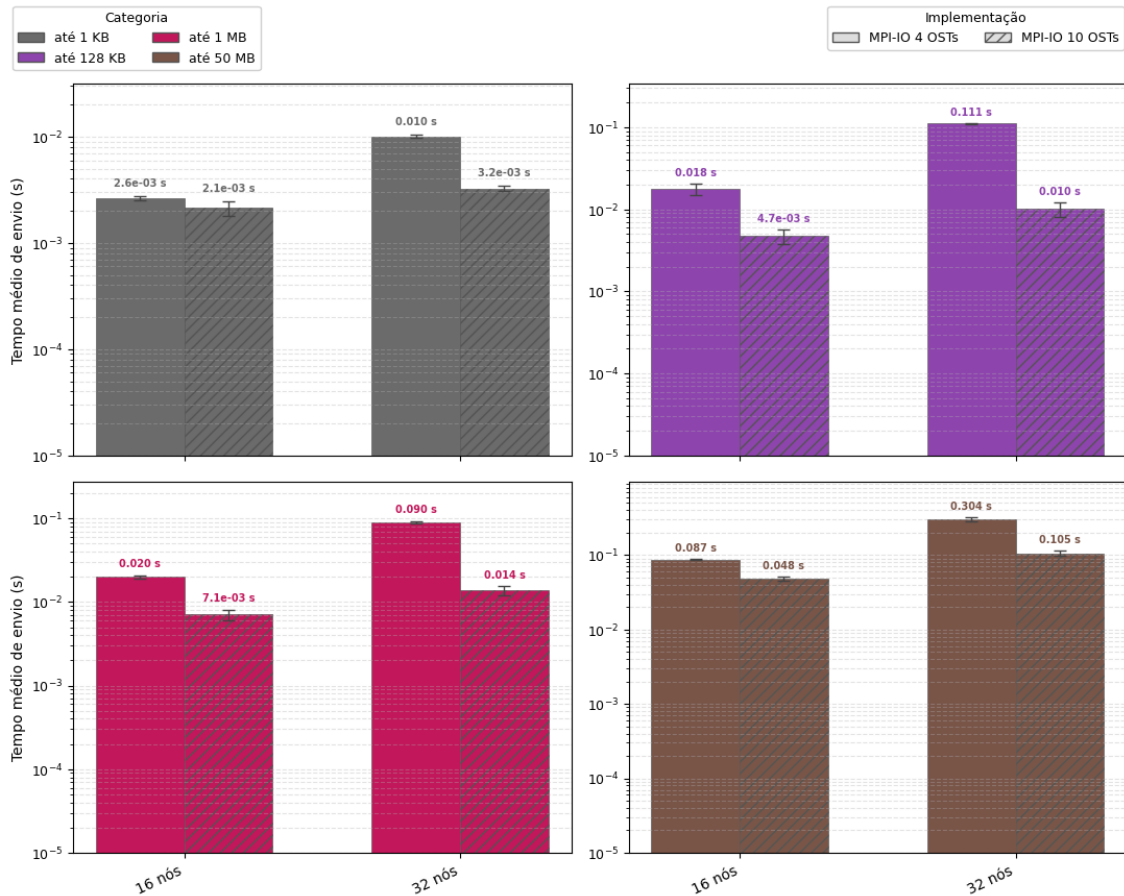


Figura 4.8: Tempo médio de envio por classe de tamanho das mensagens para configurações MPI-IO com 4 e 10 OSTs.

A Figura 4.9 complementa essa análise ao apresentar a banda agregada média obtida em cada configuração. Com 16 nós, a configuração com 10 OSTs atinge 96,2 Gb/s, contra 78,1 Gb/s com 4 OSTs. Com 32 nós, a diferença aumenta substancialmente: a banda agregada chega a 143,0 Gb/s com 10 OSTs, enquanto a configuração com 4 OSTs fica limitada a 36,7 Gb/s. Esse comportamento indica que, quando a concorrência entre processos aumenta, a configuração com menos OSTs passa a limitar a escalabilidade da E/S.

Assim, a análise conjunta das duas figuras indica que o aumento no número de OSTs melhora tanto a largura de banda agregada quanto o tempo médio das operações de envio. A configuração com 10 OSTs permite melhor distribuição das requisições entre os alvos de armazenamento, enquanto a configuração com 4 OSTs concentra o tráfego em menos dispositivos e introduz gargalos mais visíveis no cenário com 32 nós.

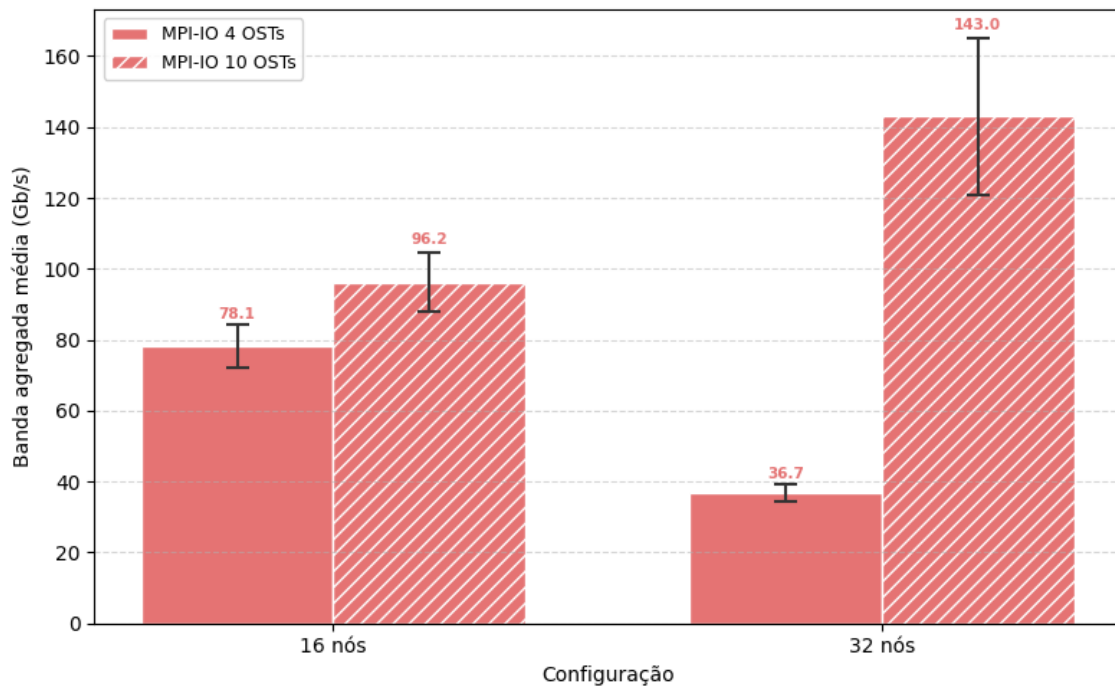


Figura 4.9: Banda agregada média para configurações MPI-IO com 4 e 10 OSTs em 16 e 32 nós.

Capítulo 5

Trabalhos relacionados

Capítulo 6

Conclusões e trabalhos futuros

Os resultados apresentados nesta dissertação evidenciam que o desempenho da aplicação SDDP é fortemente influenciado pela estratégia adotada para as operações de entrada e saída (E/S). A análise detalhada do comportamento do `MPI_Send` revelou que o tempo de bloqueio associado ao envio de blocos de dados representa um dos principais fatores limitantes da escalabilidade do modelo, sobretudo quando há um único processo responsável pela escrita em disco. Esse gargalo torna-se mais pronunciado à medida que cresce o número de nós computacionais, restringindo a sobreposição entre comunicação e computação e, consequentemente, a eficiência paralela da aplicação.

A adoção de uma arquitetura de E/S MPI-IO, baseada em MPI-IO e no sistema de arquivos Lustre, mostrou-se uma alternativa promissora para mitigar esses efeitos. Essa abordagem permite que múltiplos processos MPI realizem operações de escrita de forma paralela e coordenada, eliminando a dependência de um processo escritor único e aproveitando o paralelismo nativo do sistema de arquivos. Além de reduzir o tempo de bloqueio dos processos, essa solução potencializa o uso da largura de banda agregada e melhora a escalabilidade em aplicações com alto volume de dados.

Como trabalhos futuros, propõe-se investigar o impacto de diferentes estratégias de agregação de escrita coletiva (*two-phase I/O*) e o uso de *hints* do ROMIO para otimizar o alinhamento das operações de escrita com a configuração de *striping* do Lustre. Tais aprimoramentos poderão contribuir para a consolidação de um modelo de E/S escalável e portátil, aplicável a uma ampla gama de aplicações científicas e de engenharia com padrões intensivos de acesso a dados.

Referências Bibliográficas

- KANG, Q., LEE, S., HOU, K.-Y., et al. “Improving MPI Collective I/O for High Volume Non-Contiguous Requests With Intra-Node Aggregation”, *IEEE Transactions on Parallel and Distributed Systems*, v. 31, n. 11, pp. 2682–2695, 2020. doi: 10.1109/TPDS.2020.3000458.
- LUU, H., WINSLETT, M., GROPP, W., et al. “A Multiplatform Study of I/O Behavior on Petascale Supercomputers”. In: *Proceedings of the 24th International Symposium on High-Performance Parallel and Distributed Computing*, HPDC '15, pp. 33–44, New York, NY, USA, 2015. Association for Computing Machinery. doi: 10.1145/2749246.2749269.
- XIE, B., ORAL, S., ZIMMER, C., et al. “Characterizing Output Bottlenecks of a Production Supercomputer: Analysis and Implications”, *ACM Transactions on Storage*, v. 15, n. 4, pp. 1–39, 2020. doi: 10.1145/3335205.
- MESSAGE PASSING INTERFACE FORUM. “MPI: A Message-Passing Interface Standard, Version 5.0”. 2025. <https://www.mpi-forum.org/docs/>.
- OPENSFS AND EOFS. *Lustre Software Release 2.x: Operations Manual*, 2025. https://doc.lustre.org/lustre_manual.pdf.
- CONEJO, A. J., CARRIÓN, M., MORALES, J. M. *Decision Making Under Uncertainty in Electricity Markets*, v. 153, *International Series in Operations Research & Management Science*. New York, NY, Springer, 2010. ISBN: 9781441974211. doi: 10.1007/978-1-4419-7421-1.
- ZAKARIA, A., ISMAIL, F. B., LIPU, M. S. H., et al. “Uncertainty Models for Stochastic Optimization in Renewable Energy Applications”, *Renewable Energy*, v. 145, pp. 1543–1571, 2020. doi: 10.1016/j.renene.2019.07.081.
- PEREIRA, M. V. F., PINTO, L. M. V. G. “Multi-stage Stochastic Optimization Applied to Energy Planning”, *Mathematical Programming*, v. 52, n. 1–3, pp. 359–375, 1991.

- MESSAGE PASSING INTERFACE FORUM. “MPI: A Message-Passing Interface Standard, Version 4.1”. 2023. <https://www.mpi-forum.org/>.
- MESSAGE PASSING INTERFACE FORUM. “MPI-2: Extensions to the Message-Passing Interface”. 1997. Specification.
- GROPP, W., LUSK, E., SKJELLUM, A. *Using MPI: Portable Parallel Programming with the Message-Passing Interface*. 2 ed. Cambridge, MA, MIT Press, 1999. ISBN: 9780262571326.
- THAKUR, R., RABENSEIFNER, R., GROPP, W. “Optimization of Collective Communication Operations in MPICH”, *The International Journal of High Performance Computing Applications*, v. 19, n. 1, pp. 49–66, 2005. doi: 10.1177/1094342005051521.
- ROSS, R., THAKUR, R., GROPP, W. *Parallel I/O for High Performance Computing*. Burlington, MA, Morgan Kaufmann, 2010.
- THAKUR, R., GROPP, W., LUSK, E. “Data Sieving and Collective I/O in ROMIO”. In: *Proceedings of the 7th Symposium on the Frontiers of Massively Parallel Computation*, pp. 182–189, 1999a.
- CIRNE, W., BRASILEIRO, F., SAUVÉ, J., et al. “Grid Computing for Bag of Tasks Applications”. In: *Proceedings of the 3rd IFIP Conference on E-Commerce, E-Business and E-Government (I3E)*, 2003.
- BRAAM, P. J. “The Lustre Storage Architecture”, *Cluster File Systems Inc.*, 2002.
- PETRINI, F., KERBYSON, D. J., PAKIN, S. “The Case of the Missing Supercomputer Performance: Achieving Optimal Performance on the 8,192 Processors of ASCI Q”. In: *Proceedings of the 2003 ACM/IEEE Conference on Supercomputing*, p. 55. ACM, 2003. doi: 10.1145/1048935.1050204.
- GRAHAM, R. L., SHIPMAN, G. M., BARRETT, B. W., et al. “Open MPI: A High-Performance, Heterogeneous MPI”. In: *Proceedings of the 5th International Workshop on Algorithms, Models and Tools for Parallel Computing on Heterogeneous Networks*, 2005.
- JACKSON, K. R., RAMAKRISHNAN, L., MURIKI, K., et al. “Performance Analysis of High Performance Computing Applications on the Amazon Web Services Cloud”. In: *Proceedings of the 2010 IEEE Second International Conference on Cloud Computing Technology and Science*, pp. 159–168. IEEE, 2010. doi: 10.1109/CloudCom.2010.69.

- TRAN, A., THEKKEDATH, B. “Running Simcenter STAR-CCM+ on AWS with AWS ParallelCluster, Elastic Fabric Adapter and Amazon FSx for Lustre”. AWS Compute Blog, 2020. Disponível em: <<https://aws.amazon.com/blogs/compute/running-simcenter-star-ccm-on-aws/>>.
- OPERADOR NACIONAL DO SISTEMA ELÉTRICO. “Programa Mensal da Operação — PMO”. <https://www.ons.org.br/>, 2023.
- THAKUR, R., GROPP, W., LUSK, E. “On Implementing MPI-IO Portably and with High Performance”, *Proceedings of the 6th Workshop on I/O in Parallel and Distributed Systems*, pp. 23–32, 1999b.
- CARNS, P., LATHAM, R., ROSS, R., et al. “24/7 Characterization of Petascale I/O Workloads”. In: *Proceedings of the 2009 IEEE International Conference on Cluster Computing and Workshops*, pp. 1–10. IEEE, 2009. doi: 10.1109/CLUSTER.2009.5289150.
- AMDAHL, G. M. “Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities”. In: *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference, AFIPS '67 (Spring)*, pp. 483–485, New York, NY, USA, 1967. Association for Computing Machinery. doi: 10.1145/1465482.1465560.
- INTEL CORPORATION. “Intel Xeon Scalable Processors — 4th Generation (Sapphire Rapids)”. <https://www.intel.com/content/www/us/en/products/details/processors/xeon/scalable.html>, 2023.
- AMAZON WEB SERVICES. “Amazon EC2 C7i Instances — Compute Optimized”. <https://aws.amazon.com/ec2/instance-types/c7i/>, 2024.